

수업 06 · 전체 과정 11/20일차

# HTML/CSS/React/MCP

**2026-10-21 (수) · 10:00~17:00**

50분 수업 · 10분 휴식 · 중식 11:50~13:00

HTML과 CSS로 화면을 읽고 고친 뒤 React의 화면 부품을 살펴봅니다. 별도 MCP 예제로 도구의 요청과 응답을 확인합니다.

| 교안 읽기              | 직접 해보기              | 결과 확인하기             |
|--------------------|---------------------|---------------------|
| 개념과 완성 예제를 먼저 봅니다. | 필수 활동을 열어 작성·수정합니다. | 예상과 실제를 비교하고 설명합니다. |

## 처음 배우는 사람을 위한 수업

파일과 폴더, 저장과 실행부터 설명합니다. 설치·계정 문제가 있으면 제공 예제의 수동 추적이나 강사 시연으로 같은 개념을 확인하고, 실제 실행 여부를 구분해 기록합니다.

[웹 강의실 열기](#) · 화면의 링크와 내려받기 자료를 함께 사용하세요.

PDF 제작일 2026-10-05 · 원본 웹 내용 + 50분 교시 운영·설명 보완

# 이 교안의 순서

|                     |    |
|---------------------|----|
| 오늘 준비할 것            | 3  |
| 오늘의 6교시             | 4  |
| 1교시 · 웹 문서의 내용      | 5  |
| 2교시 · CSS의 모양과 읽기   | 11 |
| 3교시 · 버튼과 데이터 변화    | 18 |
| 4교시 · React 완성 부품   | 24 |
| 5교시 · MCP 요청과 응답    | 31 |
| 6교시 · 역할 연결과 검수     | 37 |
| 추가 활동 · 필수 경로를 마친 뒤 | 44 |
| 용어와 코드 기호 찾아보기      | 46 |
| 오늘 배운 내용 확인하기       | 47 |
| 파일과 다음 수업           | 49 |

# 오늘 준비할 것

## 처음 시작하기 · HTML/CSS/React/MCP

1. ZIP을 일반 폴더에 풀고 examples/course의 data·work·code 위치를 유지합니다. 배포 ZIP은 course 폴더부터 열 수 있습니다.
2. [교안](#)에서 개념을 읽고 [실습지](#)의 A01부터 진행합니다.
3. work 파일만 수정하고 원본은 보존합니다. 같은 파일을 저장·다시 열어 확인합니다.
4. 개인 실행을 못 했으면 예상과 강사 실행 결과를 대조하고 확인 방법을 정확히 적습니다.

### 실행 준비

브라우저와 텍스트 편집기가 기본입니다. Python 명령의 현재 폴더는 압축을 푼 course입니다. Windows에서 python 대신 py만 있으면 강사의 확인 아래 py -3로 대체합니다.

```
python code/mcp_client.py
```

실행하지 못하면 파일 경로·명령·오류 문구를 기록해 도움을 요청합니다. 코드 상자를 보는 것은 Python 실행이 아닙니다.

React 19.1.1과 React DOM 19.1.1을 로컬 번들로 제공합니다. 설치와 전체 앱 작성은 필수가 아닙니다. MCP는 2025-11-25 명시 버전의 소규모 STUDIO 예제이며 최신 전체 규격·HTTP·인증·AI 호스트 연결을 구현했다는 뜻이 아닙니다. React와 MCP 자동 연결은 없습니다.

### 수업 전, 이 세 가지부터 확인하세요

앞선 수업을 기억하지 못해도 아래 질문과 풀이부터 다시 읽으면 됩니다. 먼저 자기 말로 답한 뒤 풀이와 비교하세요.

#### 역할

질문: HTML, CSS, JavaScript는 같은 일을 하나요?

풀이: HTML은 내용과 구조, CSS는 모양, JavaScript는 동작을 담당합니다. 버튼 글자와 버튼 색, 클릭 뒤 값 변화를 나눠 봅니다.

#### 상태

질문: 센서 카드 값을 바꾼 뒤 새로고침하면 꼭 남나요?

풀이: 화면의 현재 상태와 저장된 데이터를 구별합니다. 저장 기능이 없다면 초기 입력으로 돌아갈 수 있습니다.

#### 연결

질문: React 화면이 있으면 MCP 도구도 자동으로 붙나요?

풀이: 화면과 도구 요청 경로는 별도로 연결하고 확인해야 합니다. 이 수업의 제공 예제는 두 역할을 구별해 읽습니다.

## 오늘의 수업과 복습을 나누는 방법

수업은 10:0010:50, 11:0011:50, 13:0013:50, 14:0014:50, 15:0015:50, 16:0016:50입니다. 중식은 11:5013:00이며 수업 사이에는 10분 쉽니다. 마지막 16:5017:00은 휴식과 저장·퇴실 준비로 배치했습니다.

| 교시 | 수업에서 먼저 확인할 활동 | 추가·복습           |
|----|----------------|-----------------|
| 1  | A01, A02, A03  | 새 입력으로 한 번 더 확인 |
| 2  | A04, A05, A06  | 새 입력으로 한 번 더 확인 |
| 3  | A07, A08, A09  | 새 입력으로 한 번 더 확인 |
| 4  | A10, A11, A12  | 새 입력으로 한 번 더 확인 |
| 5  | A13, A15       | A14             |
| 6  | A16, A17, A18  | 새 입력으로 한 번 더 확인 |

교안의 쉬운 사례와 단계별 풀이를 먼저 읽고 필수 결과를 직접 만들어 봅니다. 새 독립 문제는 결과를 확인한 뒤 복습하거나 도전합니다. 모든 페이지를 같은 시간에 끝내라는 뜻은 아닙니다. 설치가 막혔을 때는 제공 출력과 수동 추적으로 개념을 확인하고, 직접 실행했는지는 따로 적습니다.

## 오늘의 6교시

| 교시·시간              | 학습 초점        | 필수 활동         |
|--------------------|--------------|---------------|
| 1교시<br>10:00~10:50 | 문서 내용과 내부 연결 | A01, A02, A03 |
| 2교시<br>11:00~11:50 | 모양과 읽기 조건    | A04, A05, A06 |
| 3교시<br>13:00~13:50 | 클릭과 데이터의 이동  | A07, A08, A09 |
| 4교시<br>14:00~14:50 | React 부품과 상태 | A10, A11, A12 |
| 5교시<br>15:00~15:50 | 도구 요청과 응답    | A13, A15      |
| 6교시<br>16:00~16:50 | 네 역할과 확인 범위  | A16, A17, A18 |

설명·시연·함께 수행·개별 수행·폴이를 합쳐 교시당 50분입니다. 추가 활동까지 모두 같은 시간에 끝내라는 뜻은 아닙니다. 마지막 10분은 휴식·종료 준비로 배치합니다. 실제 학습자 소요시간은 수업에서 측정해 조정합니다.

이전 Python·Git 기초를 짧게 복습하며 준비된 작은 예제를 읽고 수정합니다. **최신 운영은 50분 × 6교시, 순수 수업 300분**입니다. 설명 보충은 필수 개념이며 독립 과제는 추가 복습입니다. 추가 문제는 수업 후 복습에 사용합니다.

React 19.1.1과 React DOM 19.1.1을 로컬 번들로 제공합니다. 설치와 전체 앱 작성은 필수가 아닙니다. MCP는 2025-11-25 명시 버전의 소규모 STUDIO 예제이며 최신 전체 규격·HTTP·인증·AI 호스트 연결을 구현했다는 뜻이 아닙니다. React와 MCP 자동 연결은 없습니다.

전체 코드와 모든 줄의 뜻·실행 방법은 [코드 해설](#)에 있습니다. 작성 소스와 생성된 번들·잠금 파일을 구분합니다.

## 1교시 · 웹 문서의 내용

10:00~10:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 폴이·확인 7분

**이번 교시에서 확인할 것.** 문서 내용과 내부 연결

**먼저 할 활동:** A01, A02, A03

이 교시는 50분 운영이며 아래 세 개념을 연결합니다. 필수 경로를 먼저 진행하고 추가 복습은 수업 진행에 따라 배정합니다. 학생은 자신의 말로 확인 근거를 설명합니다.

**추가·복습:** 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

### A01 · 파일과 브라우저의 관계

#### 사례로 이해하기 · 문서의 의미와 화면 표시를 분리하기

##### 실무 상황

작업장 안내판의 제목을 바꿨는데 브라우저 탭 이름은 예전대로입니다. 파일 이름도 달라지지 않았습니. 제목이라는 말이 세 곳에 쓰이지만 파일 이름, 탭 이름, 본문 제목은 각각 다른 정보입니다. 어디를 바꿨고 어떤 결과를 기대했는지 먼저 정해야 합니다.

##### 쉬운 폴이

HTML은 문서 내용과 의미를 표시하는 글입니다. 태그는 내용을 감싸며 역할을 붙이는 표시입니다. h1은 본문의 큰 제목 역할이고 title은 브라우저 탭의 이름입니다. 속성은 태그에 붙이는 추가 정보이며 id는 프로그램이 특정 요소를 찾는 이름입니다. 편집기는 글을 고치는 곳, 브라우저는 문서를 읽어 화면으로 보여 주는 곳입니다. 확장자 .html은 파일 종류를 나타내며 .html.txt와 구별합니다.

##### 완성 예와 단계 폴이

아래 문서를 안내.html로 저장합니다. 꺾쇠와 닫는 태그를 유지하고 한글 내용만 읽습니다.

```
<!doctype html>

<html lang="ko"><head><meta charset="utf-8"><title>작업장</title></head>

<body><h1>센서 안내</h1><button id="check">값 확인</button></body></html>
```

첫 줄은 HTML 문서 형식, lang은 한국어 정보, charset은 한글을 읽는 문자 규칙입니다. head는 문서 정보, body는 보이는 내용입니다. 브라우저에서 탭은 작업장, 본문은 센서 안내, 버튼은 값 확인으로 보입니다. 버튼은 아직 동작 코드가 없으므로 누른다고 센서 값을 읽지 않습니다. 저장한 같은 파일을 열고 수정 뒤 새로고침합니다.

## 독립 추가 복습 과제

탭은 "공장 A", 본문 제목은 "온도 확인", 버튼은 "기록 조회"로 작성하되 버튼 id는 check를 유지하세요. 이어서 id까지 바꾸면 뒤의 JavaScript 연결에 어떤 확인이 필요한지 설명하세요.

## 답과 대표 오답

title과 h1과 버튼 사이 글자를 각각 바꾸며 id는 유지합니다. "글자 크기가 같으면 제목 역할도 같다"는 답은 의미와 모양을 혼동합니다. "버튼이 보이면 데이터 연결도 됐다"는 답에는 동작 확인이 빠졌습니다.

## 확인 질문과 해설

① 저장만 하면 화면이 자동 갱신되나요? 제공 정적 파일은 새로고침이 필요합니다. ② id는 사용자 문구인가요? 내부에서 찾는 이름입니다. ③ 한글이 깨지면 무엇을 확인하나요? 파일 저장 인코딩과 charset, 실제 열린 경로를 확인합니다.

**뜻과 비유.** 브라우저는 HTML 문서를 읽어 화면으로 보여 주는 프로그램입니다. 파일을 편집하는 창과 결과를 보는 창은 역할이 다릅니다.

**왜 필요한가.** 어디에 썼는지와 무엇을 열었는지를 구분해야 저장한 변경을 다시 확인할 수 있습니다.

**예를 따라 읽기.** work/static/index.html을 편집하고 같은 파일을 브라우저로 열면 제공 안내판을 볼 수 있습니다.

**헛갈리는 반례.** 코드 편집기에 글자를 바꾼 것만으로 이미 열린 화면이 자동 갱신되는 것은 아닙니다.

**직접 적용.** course 폴더를 압축 풀기 → work/static/index.html 경로 적기 → 브라우저에서 안내판 열기 → 편집·저장·새로고침 순서를 말하기

**설명할 질문.** 편집 창과 브라우저 화면은 어떻게 다른가요?

**막혔을 때.** 확장자가.txt로 붙었으면 파일 이름과 종류를 확인합니다. ZIP 안의 미리 보기에서 편집하지 않습니다.

입력·답칸·확인 순서는 [실습지의 A01](#)과 [활동 원문](#)에 있습니다.

## A01 · 따라 하기와 작성칸

## A01 · 파일과 브라우저의 관계

### 목적

어디에 썼는지와 무엇을 열었는지를 구분해야 저장한 변경을 다시 확인할 수 있습니다.

## 준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [index.html](#), [index.html](#), [A01\\_input.md](#)

작업: [A01\\_notes.md](#), [index.html](#)

## 실제 입력

work/static/index.html을 편집하고 같은 파일을 브라우저로 열면 제공 안내판을 볼 수 있습니다.

## 단계

1. course 폴더를 압축 풀기
2. work/static/index.html 경로 적기
3. 브라우저에서 안내판 열기
4. 편집·저장·새로고침 순서를 말하기

## 예상 결과

안내판 화면과 정확한 파일 경로

## 확인 방법

편집 창과 브라우저 화면은 어떻게 다른가요?

## 오류 해결

확장자가.txt로 붙었으면 파일 이름과 종류를 확인합니다. ZIP 안의 미리 보기에서 편집하지 않습니다.

## 힌트

브라우저는 HTML 문서를 읽어 화면으로 보여 주는 프로그램입니다. 파일을 편집하는 창과 결과를 보는 창은 역할이 다릅니다.

## 빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

## 학생 답칸

- 개념 설명: 편집 창과 브라우저 화면은 어떻게 다른가요? → \_\_\_\_
- 입력 파일과 내가 바꾼 위치: \_\_\_\_

- 실행 전 예상 결과: \_\_\_\_\_
- 실제 관찰 결과와 확인 방법: \_\_\_\_\_
- 예상과 달랐던 점·복구·남은 질문: \_\_\_\_\_

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: \_\_\_\_\_

실제 결과·근거: \_\_\_\_\_

다음에 확인할 것: \_\_\_\_\_

## A02 · HTML 태그와 문서 의미

**뜻과 비유.** HTML은 제목·문단·버튼 같은 내용의 의미를 표시하는 문서 형식입니다. 여는 태그와 닫는 태그 사이의 내용이 하나의 요소가 됩니다.

**왜 필요한가.** 보기에 큰 글자를 만드는 것과 문서의 제목이라는 의미를 주는 것을 구별해야 내용 구조를 읽을 수 있습니다.

**예를 따라 읽기.** `<h1>입구 센서 안내판</h1>`에서 h1은 주요 제목, 안의 글자는 화면 내용입니다.

**헛갈리는 반례.** 모든 글자를 h1으로 만들면 크기가 같아질 수 있어도 제목과 설명의 관계가 흐려집니다.

**직접 적용.** 전체 HTML에서 h1·p·button 찾기 → 여는 태그와 닫는 태그 짝 표시하기 → 제목 글자만 참고 센서 안내판으로 바꾸기 → 저장 후 어떤 내용이 바뀌었는지 확인하기

**설명할 질문.** h1과 CSS 글자 크기는 왜 같은 역할이 아닌가요?

**막혔을 때.** 태그 꺾쇠를 지웠다면 원본의 해당 줄과 비교합니다. 제목 문구와 title 탭 이름은 다른 위치입니다.

입력·답칸·확인 순서는 [실습지의 A02](#)와 [활동 원문](#)에 있습니다.

## A02 · 따라 하기와 작성칸

## A02 · HTML 태그와 문서 의미

### 목적

보기에 큰 글자를 만드는 것과 문서의 제목이라는 의미를 주는 것을 구별해야 내용 구조를 읽을 수 있습니다.

### 준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [index.html](#), [index.html](#), [A02\\_input.md](#)

작업: [A02\\_notes.md](#), [index.html](#)

### 실제 입력

`<h1>입구 센서 안내판</h1>`에서 h1은 주요 제목, 안의 글자는 화면 내용입니다.

## 단계

1. 전체 HTML에서 h1·p·button 찾기
2. 여는 태그와 닫는 태그 짝 표시하기
3. 제목 글자만 창고 센서 안내판으로 바꾸기
4. 저장 후 어떤 내용이 바뀌었는지 확인하기

## 예상 결과

제목이 바뀐 작업 HTML

## 확인 방법

h1과 CSS 글자 크기는 왜 같은 역할이 아닌가요?

## 오류 해결

태그 꺾쇠를 지웠다면 원본의 해당 줄과 비교합니다. 제목 문구와 title 탭 이름은 다른 위치입니다.

## 힌트

HTML은 제목·문단·버튼 같은 내용의 의미를 표시하는 문서 형식입니다. 여는 태그와 닫는 태그 사이의 내용이 하나의 요소가 됩니다.

## 빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

## 학생 답칸

- 개념 설명: h1과 CSS 글자 크기는 왜 같은 역할이 아닌가요? → \_\_\_\_\_
- 입력 파일과 내가 바꾼 위치: \_\_\_\_\_
- 실행 전 예상 결과: \_\_\_\_\_
- 실제 관찰 결과와 확인 방법: \_\_\_\_\_
- 예상과 달랐던 점·복구·남은 질문: \_\_\_\_\_

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: \_\_\_\_\_

실제 결과·근거: \_\_\_\_\_

다음에 확인할 것: \_\_\_\_\_

## A03 · 속성·연결·접근성

**뜻과 비유.** 속성은 요소에 추가 정보를 붙이는 이름과 값입니다. lang은 문서 언어, id 는 특정 요소를 찾는 이름, type은 버튼 동작 종류를 설명합니다.

**왜 필요한가.** 사람이 보는 문구와 프로그램이 찾는 이름을 구분해야 버튼 연결을 끊지 않고 내용을 수정할 수 있습니다.

**예를 따라 읽기.** id ="check"는 JavaScript가 버튼을 찾는 기준이고 버튼 안 글자는 사용자에게 보이는 안내입니다.

**헛갈리는 반례.** id를 임의로 바꾸고 연결 코드를 그대로 두면 화면은 보여도 버튼 반응이 사라질 수 있습니다.

**직접 적용.** lang-id-type 속성 찾기 → 보이는 글자와 내부 이름 나누기 → 버튼 문구만 값 다시 확인으로 바꾸기 → Tab 키로 버튼에 이동해 테두리 보기

**설명할 질문.** 사용자에게 보이는 이름과 코드가 찾는 이름은 왜 구분하나요?

**막혔을 때.** 버튼 반응이 없으면 id와 선택자 #check가 같은지 봅니다. 화면 문구를 바꾸려면 태그 사이를 고칩니다.

입력·답칸·확인 순서는 [실습지의 A03](#)와 [활동 원문](#)에 있습니다.

## A03 · 따라 하기와 작성칸

### A03 · 속성·연결·접근성

#### 목적

사람이 보는 문구와 프로그램이 찾는 이름을 구분해야 버튼 연결을 끊지 않고 내용을 수정할 수 있습니다.

#### 준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [index.html](#), [app.js](#), [index.html](#), [A03\\_input.md](#)

작업: [A03\\_notes.md](#), [index.html](#)

#### 실제 입력

id ="check"는 JavaScript가 버튼을 찾는 기준이고 버튼 안 글자는 사용자에게 보이는 안내입니다.

#### 단계

1. lang-id-type 속성 찾기
2. 보이는 글자와 내부 이름 나누기
3. 버튼 문구만 값 다시 확인으로 바꾸기
4. Tab 키로 버튼에 이동해 테두리 보기

#### 예상 결과

내용 수정과 내부 연결 이름의 차이 기록

#### 확인 방법

사용자에게 보이는 이름과 코드가 찾는 이름은 왜 구분하나요?

## 오류 해결

버튼 반응이 없으면 id와 선택자 #check가 같은지 봅니다. 화면 문구를 바꾸려면 태그 사이를 고칩니다.

## 힌트

속성은 요소에 추가 정보를 붙이는 이름과 값입니다. lang은 문서 언어, id 는 특정 요소를 찾는 이름, type은 버튼 동작 종류를 설명합니다.

## 빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

## 학생 답칸

- 개념 설명: 사용자에게 보이는 이름과 코드가 찾는 이름은 왜 구분하나요? → \_\_\_\_\_
- 입력 파일과 내가 바꾼 위치: \_\_\_\_\_
- 실행 전 예상 결과: \_\_\_\_\_
- 실제 관찰 결과와 확인 방법: \_\_\_\_\_
- 예상과 달랐던 점·복구·남은 질문: \_\_\_\_\_

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: \_\_\_\_\_

실제 결과·근거: \_\_\_\_\_

다음에 확인할 것: \_\_\_\_\_

# 2교시 · CSS의 모양과 읽기

11:00~11:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

**이번 교시에서 확인할 것.** 모양과 읽기 조건

**먼저 할 활동:** A04, A05, A06

이 교시는 50분 운영이며 아래 세 개념을 연결합니다. 필수 경로를 먼저 진행하고 추가 복습은 수업 진행에 따라 배정합니다. 학생은 자신의 말로 확인 근거를 설명합니다.

**추가·복습:** 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

## A04 · CSS 선택자와 선언

### 사례로 이해하기 · 선택자와 상자 모델로 읽기 편한 화면 만들기

#### 실무 상황

센서 카드의 글자가 테두리에 붙어 읽기 어렵습니다. 카드 사이 간격도 좁습니다. 글자만 띄우려다가 페이지 전체를 오른쪽으로 밀면 작은 화면에서 내용이 잘립니다. 어떤 상자의 어느 공간을 바꾸는지 먼저 그려 봅니다.

#### 쉬운 풀이

CSS는 문서의 모양을 정하는 규칙입니다. 선택자는 꾸밀 대상을 고르는 이름이고 선언은 속성과 값의 짝입니다. 중괄호 안의 `color: teal;`은 글자색을 청록으로 정합니다. 세미콜론은 선언의 끝입니다. 상자에서 content는 내용, padding은 내용과 테두리 사이, border는 테두리, margin은 다른 상자와의 바깥 간격입니다. 색과 여백은 온도 데이터의 의미를 바꾸지 않습니다.

#### 완성 예와 풀이

앞 문서의 head 안에 style 태그로 아래 규칙을 넣습니다.

```
body { margin: 16px; font-family: sans-serif; }
h1 { color: teal; }
button { padding: 12px; margin-top: 8px; }
button:focus-visible { outline: 3px solid black; }
```

body 규칙은 전체 가장자리와 글꼴, h1 규칙은 제목만, button 규칙은 버튼 안팎 공간을 조절합니다. 마지막 규칙은 키보드로 이동할 때 초점을 표시합니다. 저장 후 새로고침하고 Tab 키로 버튼에 이동합니다. 초점은 지금 키보드로 조작할 대상이며 색 표시만으로 정상 상태를 전달하지 않듯 초점도 보이게 제공합니다. 별도 style.css를 쓰면 HTML의 link 경로가 같은 파일을 가리키는지도 확인합니다.

#### 독립 추가 복습 과제

버튼 글자 주변 공간만 늘리고 버튼과 제목 사이 공간은 유지하려면 어떤 속성을 바꾸나요? 다음으로 정상 상태를 녹색 원 하나만으로 보여 주는 화면을 글자와 함께 표현하는 문장을 작성하세요.

#### 답과 대표 오답

안쪽 공간은 padding입니다. 상태 문장은 “정상: 제공된 가상 온도가 기준 미만입니다”처럼 뜻을 적습니다. margin을 늘리는 답은 바깥 공간을 바꿉니다. “색을 더 진하게 하면 의미가 전달된다”는 답은 색을 구별하기 어려운 상황을 해결하지 못합니다.

#### 확인 질문과 해설

① CSS 변경으로 센서값이 변하나요? 표시 모양만 달라집니다. ② 적용이 안 되면 무엇부터 보나요? 저장·경로·선택자·새로고침을 확인합니다. ③ 좁은 화면도 확인하는 이유는 무엇인가요? 읽기 폭과 넘침은 큰 화면에서 보이지 않을 수 있습니다.

**뜻과 비유.** CSS는 어떤 요소에 어떤 모양을 적용할지 쓰는 규칙입니다. 선택자는 대상을 고르고, 속성과 값은 색·크기·간격을 정합니다.

**왜 필요한가.** 내용과 모양을 나누면 같은 제목을 유지하면서 여러 화면 크기에 맞게 조정할 수 있습니다.

**예를 따라 읽기.** `h1 { color: #126d65; }`는 제목 글자의 색을 정합니다. #126d65는 색을 나타내는 표기입니다.

**헛갈리는 반례.** 색을 바꿔도 센서 온도의 데이터 값이 바뀌는 것은 아닙니다.

**직접 적용.** style.css에서 h1 규칙 찾기 → color와 값의 경계 표시하기 → 색을 #75421b로 바꾸기 → 같은 제목 내용이 유지되는지 비교하기

**설명할 질문.** CSS 값을 바꿨을 때 무엇이 유지돼야하나요?

**막혔을 때.** CSS가 적용되지 않으면 HTML의 href와 파일 이름이 일치하는지 먼저 확인합니다.

입력·답칸·확인 순서는 [실습지의 A04](#)와 [활동 원문](#)에 있습니다.

## A04 · 따라 하기와 작성칸

### A04 · CSS 선택자와 선언

#### 목적

내용과 모양을 나누면 같은 제목을 유지하면서 여러 화면 크기에 맞게 조정할 수 있습니다.

#### 준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [style.css](#), [style.css](#), [index.html](#), [A04\\_input.md](#)

작업: [A04\\_notes.md](#), [style.css](#)

#### 실제 입력

`h1 { color: #126d65; }`는 제목 글자의 색을 정합니다. #126d65는 색을 나타내는 표기입니다.

#### 단계

1. style.css에서 h1 규칙 찾기
2. color와 값의 경계 표시하기
3. 색을 #75421b로 바꾸기
4. 같은 제목 내용이 유지되는지 비교하기

#### 예상 결과

색만 달라진 제목과 수정한 규칙

#### 확인 방법

CSS 값을 바꿨을 때 무엇이 유지돼야하나요?

## 오류 해결

CSS가 적용되지 않으면 HTML의 href와 파일 이름이 일치하는지 먼저 확인합니다.

## 힌트

CSS는 어떤 요소에 어떤 모양을 적용할지 쓰는 규칙입니다. 선택자는 대상을 고르고, 속성과 값은 색·크기·간격을 정합니다.

## 빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

## 학생 답칸

- 개념 설명: CSS 값을 바꿨을 때 무엇이 유지돼야하나요? → \_\_\_\_\_
- 입력 파일과 내가 바꾼 위치: \_\_\_\_\_
- 실행 전 예상 결과: \_\_\_\_\_
- 실제 관찰 결과와 확인 방법: \_\_\_\_\_
- 예상과 달랐던 점·복구·남은 질문: \_\_\_\_\_

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: \_\_\_\_\_

실제 결과-근거: \_\_\_\_\_

다음에 확인할 것: \_\_\_\_\_

## A05 · 상자·여백·읽기 폭

**뜻과 비유.** 브라우저의 요소는 화면 공간을 차지하는 상자로 배치됩니다. padding은 안쪽 간격, margin은 바깥 간격이며 max-width는 너무 넓어지지 않게 제한합니다.

**왜 필요한가.** 문장이 화면 끝까지 길어지거나 요소가 붙어 있으면 내용을 읽기 어렵습니다. 간격도 정보 전달에 영향을 줍니다.

**예를 따라 읽기.** main의 padding:24px을 32px로 바꾸면 안내판 내용과 바깥 영역 사이의 안쪽 간격이 커집니다.

**햇갈리는 반례.** 글자를 작게 줄여 모든 것을 한 화면에 넣는 방식은 초심자의 읽기를 어렵게 할 수 있습니다.

**직접 적용.** main의 세 배치 규칙 찾기 → padding과 margin을 그림으로 구분하기 → padding 한 값을 바꿔 비교하기 → 글자 크기와 읽기 폭의 영향을 설명하기

**설명할 질문.** 안쪽 여백과 바깥 여백은 어떻게 구분하나요?

**막혔을 때.** 차이가 안 보이면 같은 파일을 저장했는지와 브라우저 새로고침 여부를 확인합니다.

입력·답칸·확인 순서는 [실습지의 A05](#)와 [활동 원문](#)에 있습니다.

## A05 · 따라하기와 작성칸

# A05 · 상자·여백·읽기 폭

### 목적

문장이 화면 끝까지 길어지거나 요소가 붙어 있으면 내용을 읽기 어렵습니다. 간격도 정보 전달에 영향을 줍니다.

### 준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [style.css](#), [A05\\_input.md](#)

작업: [A05\\_notes.md](#), [style.css](#)

### 실제 입력

main의 padding:24px을 32px로 바꾸면 안내판 내용과 바깥 영역 사이의 안쪽 간격이 커집니다.

### 단계

1. main의 세 배치 규칙 찾기
2. padding과 margin 을 그림으로 구분하기
3. padding 한 값을 바꿔 비교하기
4. 글자 크기와 읽기 폭의 영향을 설명하기

### 예상 결과

간격 수정 전후 관찰 표

### 확인 방법

안쪽 여백과 바깥 여백은 어떻게 구분하나요?

### 오류 해결

차이가 안 보이면 같은 파일을 저장했는지와 브라우저 새로고침 여부를 확인합니다.

### 힌트

브라우저의 요소는 화면 공간을 차지하는 상자로 배치됩니다. padding은 안쪽 간격, margin 은 바깥 간격이며 max-width 는 너무 넓어지지 않게 제한합니다.

### 빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

## 학생 답칸

- 개념 설명: 안쪽 여백과 바깥 여백은 어떻게 구분하나요? → \_\_\_\_\_
- 입력 파일과 내가 바꾼 위치: \_\_\_\_\_
- 실행 전 예상 결과: \_\_\_\_\_
- 실제 관찰 결과와 확인 방법: \_\_\_\_\_
- 예상과 달랐던 점·복구·남은 질문: \_\_\_\_\_

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: \_\_\_\_\_

실제 결과·근거: \_\_\_\_\_

다음에 확인할 것: \_\_\_\_\_

## A06 · 작은 화면과 상태의 표시

**뜻과 비유.** 반응형 규칙은 화면 폭 같은 조건에 따라 모양을 조정하는 CSS입니다. 색만으로 의미를 전달하지 않고 글자와 구조도 함께 사용합니다.

**왜 필요한가.** 같은 자료를 좁은 화면과 키보드로도 읽을 수 있어야 업무 확인에 사용할 수 있습니다.

**예를 따라 읽기.** max-width:600px 조건에서는 main의 바깥 간격이 8px로 바뀝니다. focus-visible은 키보드 초점을 드러냅니다.

**햇갈리는 반례.** 초록색만 보고 정상이라고 판단하게 하면 색을 구분하기 어렵거나 상태가 바뀐 경우 설명이 부족합니다.

**직접 적용.** 브라우저 폭을 넓고 좁게 바꾸기 → 줄 바꿈과 가로 넘침 확인하기 → Tab으로 버튼에 이동하기 → 상태를 글자로도 알 수 있는 지 기록하기

**설명할 질문.** 정상 상태를 색만으로 표시하면 어떤 정보가 빠질까요?

**막혔을 때.** 좁은 화면이 넘치면 고정 폭을 늘리지 말고 max-width와 여백 규칙부터 확인합니다.

입력·답칸·확인 순서는 [실습지의 A06](#)과 [활동 원문](#)에 있습니다.

## A06 · 따라 하기와 작성칸

### A06 · 작은 화면과 상태의 표시

#### 목적

같은 자료를 좁은 화면과 키보드로도 읽을 수 있어야 업무 확인에 사용할 수 있습니다.

#### 준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [style.css](#), [index.html](#), [A06\\_input.md](#)

작업: [A06\\_notes.md](#), [index.html](#)

## 실제 입력

max-width:600px 조건에서는 main의 바깥 간격이 8px로 바뀝니다. focus-visible은 키보드 초점을 드러냅니다.

## 단계

1. 브라우저 폭을 넓고 좁게 바꾸기
2. 줄 바꿈과 가로 넘침 확인하기
3. Tab으로 버튼에 이동하기
4. 상태를 글자로도 알 수 있는 지 기록하기

## 예상 결과

화면 폭·키보드·상태 문구 검수 표

## 확인 방법

정상 상태를 색만으로 표시하면 어떤 정보가 빠질까요?

## 오류 해결

좁은 화면이 넘치면 고정 폭을 늘리지 말고 max-width와 여백 규칙부터 확인합니다.

## 힌트

반응형 규칙은 화면 폭 같은 조건에 따라 모양을 조정하는 CSS입니다. 색만으로 의미를 전달하지 않고 글자와 구조도 함께 사용합니다.

## 빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

## 학생 답칸

- 개념 설명: 정상 상태를 색만으로 표시하면 어떤 정보가 빠질까요? → \_\_\_\_\_
- 입력 파일과 내가 바꾼 위치: \_\_\_\_\_
- 실행 전 예상 결과: \_\_\_\_\_
- 실제 관찰 결과와 확인 방법: \_\_\_\_\_
- 예상과 달랐던 점·복구·남은 질문: \_\_\_\_\_

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: \_\_\_\_\_

실제 결과·근거: \_\_\_\_\_

다음에 확인할 것: \_\_\_\_\_

## 3교시 · 버튼과 데이터 변화

13:00~13:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

**이번 교시에서 확인할 것.** 클릭과 데이터의 이동

**먼저 할 활동:** A07, A08, A09

이 교시는 50분 운영이며 아래 세 개념을 연결합니다. 필수 경로를 먼저 진행하고 추가 복습은 수업 진행에 따라 배정합니다. 학생은 자신의 말로 확인 근거를 설명합니다.

**추가·복습:** 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

### A07 · JavaScript와 이벤트

#### 사례로 이해하기 · 버튼 사건에서 데이터 표시까지 따라가기

##### 실무 상황

안내판은 처음에 22.5를 표시하지만 담당자가 data.js 값을 25로 바꾼 뒤에도 화면은 22.5입니다. 버튼을 누르자 25가 나타납니다. 오류라고 단정하기 전에 초기 문서와 버튼 동작의 데이터 흐름을 나누어 읽습니다.

##### 쉬운 풀이

JavaScript는 브라우저에서 동작을 수행하는 프로그램 언어입니다. 이벤트는 클릭처럼 일어난 사건이고 이벤트 처리 함수는 그 사건 때 실행할 행동입니다. `querySelector`는 이름에 맞는 문서 요소를 찾습니다. `#`은 id를 찾는 표시입니다. `textContent`는 요소의 글자 내용이며 이를 바꾸면 화면 글자가 달라집니다. 데이터 파일의 값, HTML의 처음 글자, 화면의 현재 글자는 같은 장소에 있지 않습니다.

##### 완성 예와 단계 풀이

제공 static/index.html은 data.js와 app.js를 연결합니다. data.js가 먼저 window.SENSOR에 가상 기록을 넣고 app.js가 `#check-#temperature-#status`를 찾습니다. 버튼 클릭 때 temperature 요소에 SENSOR.temperature\_c를 넣고 상태에 장소 설명을 넣습니다. 처음 HTML의 22.5는 클릭 전의 표시이며 data.js에서 25를 저장한 뒤 새로고침하고 클릭하면 25가 보입니다. script 연결 순서와 같은 폴더의 파일 경로도 확인합니다. 실제 센서에서 받은 값이라는 의미는 없습니다.

##### 독립 추가 복습 과제

데이터는 26, 처음 HTML 글자는 22.5인 카드의 “열기 직후 / 클릭 뒤 / 데이터 파일 저장 여부” 세 칸을 작성하세요. 버튼 id만 read로 바꾼 경우 필요한 연결 수정을 결정하고 어떤 행동으로 복구를 확인할지 쓰세요.

## 답과 대표 오답

처음은 22.5, 클릭 뒤는 26이며 클릭은 파일에 값을 저장하지 않습니다. id를 read로 바꾸면 JavaScript 선택자도 #read로 맞춰야 합니다. 수정 후 같은 버튼 클릭으로 결과를 확인합니다. "HTML 숫자만 고치면 이후 클릭도 바뀐다"는 답은 실제 데이터 출처를 놓쳤습니다.

## 확인 질문과 해설

① 버튼이 안 되면 무조건 데이터 문제인가요? 요소 이름이나 코드 로딩도 확인합니다. ② 새로고침과 클릭을 따로 기록하는 이유는 무엇인가요? 초기 표시와 사건 후 표시가 다릅니다. ③ 복구 후 파일 내용만 보면 충분한가요? 이전 실패 행동을 다시 수행하여 동작까지 확인해야 합니다.

**뜻과 비유.** JavaScript는 프로그램의 동작을 적는 언어입니다. 이 예제에서는 클릭이나 데이터 변화에 따라 화면을 바꿉니다. 이벤트는 버튼 클릭처럼 프로그램이 반응할 수 있는 사건입니다.

**왜 필요한가.** 정적인 문서 내용과 클릭 뒤 실행되는 동작을 나누면 화면이 왜 바뀌었는지 따라갈 수 있습니다.

**예를 따라 읽기.** `addEventListener("click", ...)`는 클릭 때 안쪽 코드를 실행하도록 연결합니다.

**헛갈리는 반례.** 버튼이 보인다는 사실만으로 클릭 동작이 연결됐다고 볼 수 없습니다.

**직접 적용.** `app.js` 전체 일곱 줄 읽기 → 버튼 선택과 클릭 연결 구간 나누기 → 누르기 전 상태 문구 예상하기 → 클릭 후 바뀐 문구를 비교하기

**설명할 질문.** 버튼 표시와 클릭 반응은 어떻게 다른가요?

**막혔을 때.** 선택한 요소가 없다는 오류면 HTML의 id와 `querySelector` 이름을 비교합니다.

입력·답칸·확인 순서는 [실습지의 A07](#)과 [활동 원문](#)에 있습니다.

## A07 · 따라 하기와 작성칸

## A07 · JavaScript와 이벤트

### 목적

정적인 문서 내용과 클릭 뒤 실행되는 동작을 나누면 화면이 왜 바뀌었는지 따라갈 수 있습니다.

### 준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [app.js](#), [index.html](#), [A07\\_input.md](#)

작업: [A07\\_notes.md](#)

### 실제 입력

`addEventListener("click", ...)`는 클릭 때 안쪽 코드를 실행하도록 연결합니다.

## 단계

1. app.js 전체 일곱 줄 읽기
2. 버튼 선택과 클릭 연결 구간 나누기
3. 누르기 전 상태 문구 예상하기
4. 클릭 후 바뀐 문구를 비교하기

## 예상 결과

클릭 전후 문구와 이벤트 흐름

## 확인 방법

버튼 표시와 클릭 반응은 어떻게 다른가요?

## 오류 해결

선택한 요소가 없다는 오류면 HTML의 id와 querySelector 이름을 비교합니다.

## 힌트

JavaScript는 프로그램의 동작을 적는 언어입니다. 이 예제에서는 클릭이나 데이터 변화에 따라 화면을 바꿉니다. 이벤트는 버튼 클릭처럼 프로그램이 반응할 수 있는 사건입니다.

## 빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

## 학생 답칸

- 개념 설명: 버튼 표시와 클릭 반응은 어떻게 다른가요? → \_\_\_\_\_
- 입력 파일과 내가 바꾼 위치: \_\_\_\_\_
- 실행 전 예상 결과: \_\_\_\_\_
- 실제 관찰 결과와 확인 방법: \_\_\_\_\_
- 예상과 달랐던 점·복구·남은 질문: \_\_\_\_\_

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: \_\_\_\_\_

실제 결과·근거: \_\_\_\_\_

다음에 확인할 것: \_\_\_\_\_

## A08 · 데이터와 화면 내용 연결

**뜻과 비유.** 데이터는 프로그램이 보여 줄 값이며 화면 요소는 그 값을 담는 자리입니다. `textContent`는 글자를 넣는 방법으로 오늘의 단순 숫자·문구 표시에 사용합니다.

**왜 필요한가.** 어느 파일의 값이 어느 화면 자리로 이동하는지 알면 작은 변경을 스스로 확인할 수 있습니다.

**예를 따라 읽기.** data.js의 temperature\_c를 24.5로 바꾼 뒤 클릭하면 온도 자리에 24.5가 표시됩니다.

**헷갈리는 반례.** HTML에 처음 적힌 22.5와 클릭 후 data.js에서 읽은 값은 출처가 다르므로 수정 위치를 구분합니다.

**직접 적용.** data.js의 세 필드 읽기 → 온도 하나를 24.5로 바꾸기 → 새로고침 직후와 클릭 후 비교하기 → 초기 HTML 값과 동적 값의 출처 쓰기

**설명할 질문.** 새로고침 직후와 클릭 뒤 값이 다른 이유는 무엇인가요?

**막혔을 때.** 값이 그대로면 클릭까지 수행했는지 봅니다. 초기 HTML 값만 확인하고 변경 실패라고 판단하지 않습니다.

입력·답칸·확인 순서는 [실습지의 A08](#)와 [활동 원문](#)에 있습니다.

## A08 · 따라 하기와 작성칸

### A08 · 데이터와 화면 내용 연결

#### 목적

어느 파일의 값이 어느 화면 자리로 이동하는지 알면 작은 변경을 스스로 확인할 수 있습니다.

#### 준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [data.js](#), [app.js](#), [index.html](#), [A08\\_input.md](#)

작업: [A08\\_notes.md](#), [data.js](#), [app.js](#), [index.html](#)

#### 실제 입력

data.js의 temperature\_c를 24.5로 바꾼 뒤 클릭하면 온도 자리에 24.5가 표시됩니다.

#### 단계

1. data.js의 세 필드 읽기
2. 온도 하나를 24.5로 바꾸기
3. 새로고침 직후와 클릭 후 비교하기
4. 초기 HTML 값과 동적 값의 출처 쓰기

#### 예상 결과

입력 24.5와 클릭 후 표시 값의 대응

#### 확인 방법

새로고침 직후와 클릭 뒤 값이 다른 이유는 무엇인가요?

## 오류 해결

값이 그대로면 클릭까지 수행했는지 봅니다. 초기 HTML 값만 확인하고 변경 실패라고 판단하지 않습니다.

## 힌트

데이터는 프로그램이 보여 줄 값이며 화면 요소는 그 값을 담는 자리입니다. `textContent`는 글자를 넣는 방법으로 오늘의 단순 숫자·문구 표시에 사용합니다.

## 빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

## 학생 답칸

- 개념 설명: 새로고침 직후와 클릭 뒤 값이 다른 이유는 무엇인가요? → \_\_\_\_\_
- 입력 파일과 내가 바꾼 위치: \_\_\_\_\_
- 실행 전 예상 결과: \_\_\_\_\_
- 실제 관찰 결과와 확인 방법: \_\_\_\_\_
- 예상과 달랐던 점·복구·남은 질문: \_\_\_\_\_

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: \_\_\_\_\_

실제 결과·근거: \_\_\_\_\_

다음에 확인할 것: \_\_\_\_\_

## A09 · 오류 위치와 최소 복구

**뜻과 비유.** 오류를 찾을 때는 증상·실제 파일·가장 최근에 바꾼 부분을 순서대로 확인합니다. 코드 전체를 다시 쓰기보다 연결 하나를 비교합니다.

**왜 필요한가.** 작은 오타를 찾는 과정은 화면이 여러 파일의 연결로 만들어진다는 사실을 보여 줍니다.

**예를 따라 읽기.** 복사본의 `id = "check"`를 `id = "checkx"`로 바꾸면 `#check` 선택과 어긋납니다. 원복하면 클릭이 돌아옵니다.

**헛갈리는 반례.** 정상 원본까지 고치면 비교 기준을 잃으므로 의도적 오류는 작업 복사본에만 만듭니다.

**직접 적용.** 복사본에서 `id` 한 글자 바꾸기 → 클릭 반응이 없는 증상 관찰하기 → HTML과 JavaScript 이름 대조하기 → 원복 후 같은 행동으로 재확인하기

**설명할 질문.** 원복 후에도 같은 행동으로 확인해야 하는 이유는 무엇인가요?

**막혔을 때.** 개발자 도구가 익숙하지 않으면 먼저 원문 이름 비교로 찾고 강사가 콘솔 오류를 보여 줍니다.

입력·답칸·확인 순서는 [실습지의 A09](#)와 [활동 원문](#)에 있습니다.

## A09 · 따라 하기와 작성칸

## A09 · 오류 위치와 최소 복구

### 목적

작은 오타를 찾는 과정은 화면이 여러 파일의 연결로 만들어진다는 사실을 보여 줍니다.

### 준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [index.html](#), [app.js](#), [A09\\_input.md](#)

작업: [A09\\_notes.md](#), [index.html](#), [app.js](#)

### 실제 입력

복사본의 id ="check"를 id ="checkx"로 바꾸면 #check 선택과 어긋납니다. 원복하면 클릭이 돌아옵니다.

### 단계

1. 복사본에서 id 한 글자 바꾸기
2. 클릭 반응이 없는 증상 관찰하기
3. HTML과 JavaScript 이름 대조하기
4. 원복 후 같은 행동으로 재확인하기

### 예상 결과

한 오류의 증상·원인·복구 기록

### 확인 방법

원복 후에도 같은 행동으로 확인해야 하는 이유는 무엇인가요?

### 오류 해결

개발자 도구가 익숙하지 않으면 먼저 원문 이름 비교로 찾고 강사가 콘솔 오류를 보여 줍니다.

### 힌트

오류를 찾을 때는 증상·실제 파일·가장 최근에 바꾼 부분을 순서대로 확인합니다. 코드 전체를 다시 쓰기보다 연결 하나를 비교합니다.

### 빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

## 학생 답칸

- 개념 설명: 원복 후에도 같은 행동으로 확인해야 하는 이유는 무엇인가요? → \_\_\_\_\_
- 입력 파일과 내가 바꾼 위치: \_\_\_\_\_
- 실행 전 예상 결과: \_\_\_\_\_
- 실제 관찰 결과와 확인 방법: \_\_\_\_\_
- 예상과 달랐던 점·복구·남은 질문: \_\_\_\_\_

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: \_\_\_\_\_

실제 결과·근거: \_\_\_\_\_

다음에 확인할 것: \_\_\_\_\_

# 4교시 · React 완성 부품

14:00~14:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

**이번 교시에서 확인할 것.** React 부품과 상태

**먼저 할 활동:** A10, A11, A12

이 교시는 50분 운영이며 아래 세 개념을 연결합니다. 필수 경로를 먼저 진행하고 추가 복습은 수업 진행에 따라 배정합니다. 학생은 자신의 말로 확인 근거를 설명합니다.

**추가·복습:** 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

## A10 · React와 컴포넌트

### 사례로 이해하기 · React 부품과 브라우저가 실행하는 파일

#### 실무 상황

센서 카드를 여러 장소에 쓰려 합니다. 카드마다 제목과 온도를 직접 복사하면 한 곳을 고칠 때 다른 곳도 찾아야 합니다. 공통 화면 부품에 장소 값을 전달하면 구조를 재사용할 수 있습니다. 다만 소스 파일과 실제 실행 파일이 다르면 소스만 고치고 화면이 바뀌었다고 기대해서는 안 됩니다.

#### 쉬운 풀이

React 컴포넌트는 화면을 만드는 함수이며 props는 바깥에서 그 부품에 전달하는 값입니다. state는 화면 부품이 기억하고 바꾸는 현재 값입니다. JSX는 JavaScript 안에서 화면 구조를 적는 문법입니다. 일반 브라우저는 JSX가 들어 있는 .jsx 소스를 그대로 실행하지 못하며 변환 단계가 필요합니다. 번들은 변환된 코드와 필요한 라이브러리를 묶어 브라우저가 읽도록 만든 파일입니다. 제공 index.html은 app.bundle.js를 실행합니다.

## 완성 예와 풀이

App.jsx의 `<SensorCard location={window.SENSOR.location} />`은 같은 부품에 장소를 전달합니다. SensorCard 안에서 `useState`는 `SENSOR.temperature_c`를 시작 값으로 기억합니다. 클릭은 `setTemperature`로 현재 값을 0.5 올립니다. 제공 데이터가 22.5라면 두 번 클릭 뒤 23.5가 보이며 새로고침하면 22.5로 돌아옵니다. 클릭이 data.js나 센서 장비에 저장한 것은 아닙니다. data.js는 HTML이 별도로 읽으므로 장소를 바꿔 저장·새로고침하면 반영됩니다. App.jsx의 문구나 증가량을 바꾸려면 번들을 다시 만드는 환경이 필요하고 오늘 필수 활동은 제공 번들을 사용합니다.

## 독립 추가 복습 과제

“장소 이름 바꾸기”, “클릭한 현재 온도 올리기”, “증가량을 1로 바꾸기”를 데이터 파일·상태 변경·소스와 번들 재생성에 각각 연결하세요. 원래 값이 24라면 세 번 클릭과 새로고침 뒤의 값도 예상하세요.

## 답과 대표 오답

장소는 data.js, 현재 값은 브라우저 state, 증가량은 App.jsx 수정과 빌드가 필요합니다. 세 번 뒤 25.5이고 새로고침 뒤 24입니다. “JSX는 HTML과 같아서 파일을 열면 된다”는 답은 변환을 빠뜨렸습니다. “클릭값이 저장됐다”는 답은 메모리와 파일을 혼동합니다.

## 확인 질문과 해설

① props와 state는 같은가요? 전달 입력과 부품의 현재 기억입니다. ② 번들을 직접 고치나요? 생성 결과보다 소스를 수정하고 다시 만드는 경로가 기준입니다. ③ React가 MCP를 자동 호출하나요? 별도 연결 처리 없이는 호출하지 않습니다.

**뜻과 비유.** React는 데이터와 상태를 이용해 화면을 구성하는 JavaScript 라이브러리입니다. 컴포넌트는 내용과 동작을 함께 설명하는 재사용 가능한 화면 부품입니다.

**왜 필요한가.** 같은 센서 안내 형식을 여러 곳에 쓸 때 각 부품의 입력과 상태를 구분할 수 있습니다.

**예를 따라 읽기.** SensorCard는 장소와 온도를 보여 주는 제공된 완성 부품입니다. App은 제목과 이 부품을 화면에 배치합니다.

**헛갈리는 반례.** React는 HTML이나 CSS를 없애는 도구가 아니며 화면의 내용과 모양을 여전히 이해해야 합니다.

**직접 적용.** code/react/index.html 실행하기 → App.jsx에서 두 함수 이름 찾기 → 제목·SensorCard·안내문 위치 연결하기 → 정적 화면과 React 부품의 공통점 적기

**설명할 질문.** 컴포넌트는 표의 데이터 한 행과 같은 말인가요?

**막혔을 때.** 패키지 설치가 없어도 제공 app.bundle.js로 화면을 볼 수 있습니다. JSX를 브라우저가 그대로 읽는다고 가정하지 않습니다.

입력·답칸·확인 순서는 [실습지의 A10](#)와 [활동 원문](#)에 있습니다.

## A10 · 따라 하기와 작성칸

# A10 · React와 컴포넌트

### 목적

같은 센서 안내 형식을 여러 곳에 쓸 때 각 부품의 입력과 상태를 구분할 수 있습니다.

### 준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [App.jsx](#), [index.html](#), [A10\\_input.md](#)

작업: [A10\\_notes.md](#)

### 실제 입력

SensorCard는 장소와 온도를 보여 주는 제공된 완성 부품입니다. App은 제목과 이 부품을 화면에 배치합니다.

### 단계

1. code/react/index.html 실행하기
2. App.jsx에서 두 함수 이름 찾기
3. 제목·SensorCard·안내문 위치 연결하기
4. 정적 화면과 React 부품의 공통점 적기

### 예상 결과

완성 React 화면과 부품 역할 설명

### 확인 방법

컴포넌트는 표의 데이터 한 행과 같은 말인가요?

### 오류 해결

패키지 설치가 없어도 제공 app.bundle.js로 화면을 볼 수 있습니다. JSX를 브라우저가 그대로 읽는다고 가정하지 않습니다.

### 힌트

React는 데이터와 상태를 이용해 화면을 구성하는 JavaScript 라이브러리입니다. 컴포넌트는 내용과 동작을 함께 설명하는 재사용 가능한 화면 부품입니다.

### 빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

## 학생 답칸

- 개념 설명: 컴포넌트는 표의 데이터 한 행과 같은 말인가요? → \_\_\_\_\_
- 입력 파일과 내가 바꾼 위치: \_\_\_\_\_
- 실행 전 예상 결과: \_\_\_\_\_
- 실제 관찰 결과와 확인 방법: \_\_\_\_\_
- 예상과 달랐던 점·복구·남은 질문: \_\_\_\_\_

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: \_\_\_\_\_

실제 결과·근거: \_\_\_\_\_

다음에 확인할 것: \_\_\_\_\_

## A11 · props와 제공 데이터 수정

**뜻과 비유.** props는 부모가 부품에 전달하는 입력입니다. 오늘은 장소를 전달하고 온도의 첫 값을 제공 데이터에서 읽는 작은 완성 예제를 사용합니다.

**왜 필요한가.** 부품을 처음부터 새로 만들지 않아도 어떤 입력을 바꾸면 어느 화면이 달라지는지 확인할 수 있습니다.

**예를 따라 읽기.** work/react/data.js에서 location을 참고, temperature\_c를 24.5로 바꾸고 새로고침하면 부품의 내용이 바뀝니다.

**햇갈리는 반례.** JSX의 중괄호는 여기서 JavaScript 값을 넣는 자리이며 CSS 규칙의 중괄호와 역할이 다릅니다.

**직접 적용.** 작업 React 폴더 열기 → data.js의 장소와 온도 수정하기 → 새로고침 해두 표시 비교하기 → props로 전달되는 장소 경로 쓰기

**설명할 질문.** 같은 부품을 다른 장소에 쓰려면 어떤 입력을 바꿀까요?

**막혔을 때.** 수정한 data.js가 code인지 work인지 확인합니다. 변경 값을 읽는 곳은 현재 열린 HTML 과 같은 폴더입니다.

입력·답칸·확인 순서는 [실습지의 A11](#)와 [활동 원문](#)에 있습니다.

## 작업 화면으로 전환하기 · A10 → A11

탐색기에서 [course/work/react/index.html](#)을 브라우저로 엽니다. 주소에 **work/react/index.html**이 있는지 확인하고 기록합니다. 같은 폴더의 [data.js](#)를 수정·저장한 뒤 이 작업 탭을 새로고침합니다. 장소 [참고](#), 온도 [24.5](#)가 보이는지 비교합니다. A10의 code/react 탭은 원본이므로 작업 수정이 반영되지 않습니다. App.jsx 재빌드는 선택입니다.

## A11 · 따라 하기와 작성칸

## A11 · props와 제공 데이터 수정

### 목적

부품을 처음부터 새로 만들지 않아도 어떤 입력을 바꾸면 어느 화면이 달라지는지 확인할 수 있습니다.

## 준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [data.js](#), [App.jsx](#), [A11\\_input.md](#)

작업: [A11\\_notes.md](#), [data.js](#), [작업 index.html](#)

## 실제 입력

work/react/data.js에서 location을 참고,temperature\_c를 24.5로 바꾸고 새로고침하면 부품의 내용이 바뀝니다.

## 단계

1. 작업 React 폴더 열기
2. data.js의 장소와 온도 수정하기
3. 새로고침 해두 표시 비교하기
4. props로 전달되는 장소 경로 쓰기

## 예상 결과

장소·온도 수정본과 결과

## 확인 방법

같은 부품을 다른 장소에 쓰려면 어떤 입력을 바꿀까요?

## 오류 해결

수정한 data.js가 code인지 work인지 확인합니다. 변경 값을 읽는 곳은 현재 열린 HTML 과 같은 폴더입니다.

## 힌트

props는 부모가 부품에 전달하는 입력입니다. 오늘은 장소를 전달하고 온도의 첫 값을 제공 데이터에서 읽는 작은 완성 예제를 사용합니다.

## 빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

## 학생 답칸

- 개념 설명: 같은 부품을 다른 장소에 쓰려면 어떤 입력을 바꿀까요? → \_\_\_\_
- 입력 파일과 내가 바꾼 위치: \_\_\_\_
- 실행 전 예상 결과: \_\_\_\_
- 실제 관찰 결과와 확인 방법: \_\_\_\_
- 예상과 달랐던 점·복구·남은 질문: \_\_\_\_

## 작업 화면으로 전환하기 · A10 → A11

1. A10에서 본 `code/react/index.html` 화면을 닫거나 원본 탭이라고 표시합니다.
2. 탐색기에서 `course/work/react/index.html`을 브라우저로 엽니다. 주소 표시줄에 **work/react/index.html**이 포함됐는지 확인하고 실제 주소를 답칸에 적습니다.
3. 같은 `work/react` 폴더의 `data.js`를 편집기에서 열어 장소와 온도만 수정하고 저장합니다.
4. 2단계의 작업 화면을 새로고침합니다. 장소 **창고**, 온도 **24.5**가 보이는지 비교합니다.

원본 탭을 새로고침하면 작업값이 반영되지 않습니다. 두 파일 모두 이름은 `index.html`이므로 주소의 **code / work**까지 확인합니다. 소스 `App.jsx` 재빌드는 선택 확장입니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: \_\_\_\_\_

실제 결과·근거: \_\_\_\_\_

다음에 확인할 것: \_\_\_\_\_

## A12 · 상태와 화면의 다시 그리기

**뜻과 비유.** 상태는 컴포넌트가 기억하는 값입니다. `useState`는 현재 값과 그 값을 바꾸는 함수를 제공하며 상태 변경에 따라 화면이 다시 계산됩니다.

**왜 필요한가.** 화면 글자를 직접 찾는 방식과 상태를 바꿔 화면을 설명하는 방식의 차이를 작은 클릭으로 관찰합니다.

**예를 따라 읽기.** 0.5 올린 모의값 버튼을 누르면 22.5 → 23.0처럼 상태가 바뀝니다. 이것은 실제 센서 측정이 아닙니다.

**헷갈리는 반례.** 새로고침하면 서버에 저장한 값이 아니라 `data.js`의 초기 값으로 돌아갑니다.

**직접 적용.** `useState`와 `setTemperature` 위치 찾기 → 클릭 전후 숫자 예상하기 → 두 번 클릭한 값과 실제 비교하기 → 새로고침 뒤 초기화 이유 설명하기

**설명할 질문.** 상태 변경과 파일 저장은 어떤 차이가 있나요?

**막혔을 때.** 버튼을 누른 값이 파일에 저장됐다고 생각되면 `data.js` 원문을 다시 열어 비교합니다.

입력·답칸·확인 순서는 [실습지의 A12](#)와 [활동 원문](#)에 있습니다.

## A12 · 따라 하기와 작성칸

## A12 · 상태와 화면의 다시 그리기

### 목적

화면 글자를 직접 찾는 방식과 상태를 바꿔 화면을 설명하는 방식의 차이를 작은 클릭으로 관찰합니다.

## 준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [App.jsx](#), [index.html](#), [A12\\_input.md](#)

작업: [A12\\_notes.md](#), [index.html](#)

## 실제 입력

0.5 올린 모의값 버튼을 누르면 22.5 → 23.0처럼 상태가 바뀝니다. 이것은 실제 센서 측정이 아닙니다.

## 단계

1. useState와 setTemperature 위치 찾기
2. 클릭 전후 숫자 예상하기
3. 두 번 클릭한 값과 실제 비교하기
4. 새로고침 뒤 초기화 이유 설명하기

## 예상 결과

상태 변화와 초기화 기록

## 확인 방법

상태 변경과 파일 저장은 어떤 차이가 있나요?

## 오류 해결

버튼을 누른 값이 파일에 저장됐다고 생각되면 data.js 원문을 다시 열어 비교합니다.

## 힌트

상태는 컴포넌트가 기억하는 값입니다. useState는 현재 값과 그 값을 바꾸는 함수를 제공하며 상태 변경에 따라 화면이 다시 계산됩니다.

## 빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

## 학생 답칸

- 개념 설명: 상태 변경과 파일 저장은 어떤 차이가 있나요? → \_\_\_\_\_
- 입력 파일과 내가 바꾼 위치: \_\_\_\_\_

- 실행 전 예상 결과: \_\_\_\_\_
- 실제 관찰 결과와 확인 방법: \_\_\_\_\_
- 예상과 달랐던 점·복구·남은 질문: \_\_\_\_\_

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: \_\_\_\_\_

실제 결과·근거: \_\_\_\_\_

다음에 확인할 것: \_\_\_\_\_

## 5교시 · MCP 요청과 응답

15:00~15:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

**이번 교시에서 확인할 것.** 도구 요청과 응답

**먼저 할 활동:** A13, A15

이 교시는 50분 운영이며 아래 세 개념을 연결합니다. 필수 경로를 먼저 진행하고 추가 복습은 수업 진행에 따라 배정합니다. 학생은 자신의 말로 확인 근거를 설명합니다.

추가·복습: A14 · 필수 결과를 확인한 뒤 진행합니다.

### A13 · MCP의 호스트·클라이언트·서버

**사례로 이해하기 · 직접 연결과 MCP 연결을 비교한 뒤 권한 정하기**

#### 실무 상황

온도 안내판이 기록을 보여 주는 방법에는 파일을 직접 읽는 경로, 서버 API를 호출하는 경로, AI 도구가 MCP 서버를 통해 기록을 조회하는 경로가 있습니다. 화면 모양은 비슷해도 중간 프로그램과 권한은 다릅니다. 연결을 그렸다는 사실만으로 실제 호출이 구현되었다고 보고하지 않습니다.

#### 쉬운 풀이

MCP는 AI 애플리케이션이 도구와 자료에 접근할 때 주고받는 메시지의 공통 약속입니다. 호스트는 AI 애플리케이션, 클라이언트는 서버와 통신하는 부분, 서버는 도구와 자료를 제공하는 프로그램입니다. 직접 파일 읽기는 공통 프로토콜 없이 해당 파일 구조를 알아야 합니다. MCP는 초기화·도구 목록·호출 같은 약속을 공유하지만 모든 서버가 같은 자료나 권한을 갖지는 않습니다. 연결 가능성과 쓰기 허용도 별개입니다.

## 완성 예와 단계 풀이

오늘은 mcp\_client.py가 로컬 mcp\_server.py를 실행하여 표준 입력과 표준 출력으로 메시지를 주고받습니다. initialize 요청에 id 1이 있고 initialized 알림에는 id가 없습니다. tools/list가 get\_reading의 입력 이름 sensor\_id를 알려 줍니다. tools/call로 S01을 요청하면 22.5가 담긴 가상 기록이 돌아옵니다. S02는 존재하지만 미측정 null이며 S99는 Unknown sensor와 isError입니다. 이 서버는 읽기 도구만 제공하며 네트워크 요청, React 갱신, 실제 AI 추론을 수행하지 않습니다. 실제 호스트의 권한 통제 전체를 재현한 프로그램도 아닙니다.

## 독립 추가 복습 과제

“HTML의 고정 숫자”, “data.js의 가상 값”, “MCP get\_reading의 응답”을 자료 출처와 필요한 연결 단계로 비교하세요. 이어서 센서 조회에 계정 삭제 도구 권한까지 요구하는 서버가 있다면 허용 범위를 어떻게 판단할지 쓰세요.

## 답과 대표 오답

고정 숫자는 HTML, 가상 값은 데이터 파일, MCP 응답은 서버 기록과 호출 연결입니다. 조회 목적에 필요하지 않은 삭제 권한은 제공하지 않고 필요한 범위를 확인합니다. “MCP가 연결되면 모든 작업이 자동 허용”은 오답입니다.

## 확인 질문과 해설

① 알림에 응답 id가 있나요? 오늘 initialized 알림은 응답을 기대하지 않습니다. ② null은 도구 오류인가요? 존재하는 기록의 미측정 값일 수 있습니다. ③ 연결 증거는 무엇인가요? 실제 요청과 대응 응답, 실행 범위를 남겨야 합니다.

호스트는 사용자가 요청하는 AI 앱 전체이고, 클라이언트는 그 안에서 특정 서버와 메시지를 주고받는 연결 부분입니다. 서버는 도구와 입력 규칙을 제공하고 실행 결과를 돌려줍니다. 오늘 예제의 클라이언트는 미리 쓴 요청만 보내며 실제 AI 호스트나 모델이 판단하는 과정을 구현하지 않습니다.

**뜻과 비유.** MCP는 AI 앱이 외부 도구나 자료를 정해진 메시지 형식으로 사용하도록 연결하는 프로토콜입니다. 서버는 기능을 제공하고 클라이언트는 요청을 전달합니다.

**왜 필요한가.** React의 화면 구성과 MCP의 도구 통신을 같은 역할로 생각하지 않도록 경계를 구분합니다.

**예를 따라 읽기.** 제공 클라이언트는 get\_reading이라는 읽기 도구를 요청하고 서버는 가상 센서 기록을 반환합니다.

**헛갈리는 반례.** 오늘의 mcp\_client.py는 저장된 요청을 보내는 프로그램이며 AI 모델이나 실제 AI 호스트가 아닙니다.

**직접 적용.** 역할 카드의 네 요소 읽기 → 화면 역할과 도구 통신 역할 나누기 → 클라이언트·서버·데이터 파일 연결하기 → 실제 AI 호출이 없는 조건 쓰기

**설명할 질문.** MCP 서버가 웹 화면을 자동으로 만드는 것은 왜 아닌가요?

**막혔을 때.** MCP를 모든 AI 서비스의 동의어로 보지 않고 오늘 메시지를 주고 받는 두 프로그램을 정확히 지정합니다.

입력·답칸·확인 순서는 [실습지의 A13](#)와 [활동 원문](#)에 있습니다.

## MCP 필수 읽기와 참고 범위

**필수:** 역할 카드에서 호스트·클라이언트·서버·데이터를 구별합니다. requests.jsonl 첫 세 줄의 초기화·알림·도구 목록과 네 번째 줄의 method/id/params를 읽습니다. recorded\_responses.json에서 S01의 숫자, S02의 미측정, S99의 isError를 대조합니다. 제공 클라이언트 실행 또는 강사 실행 관찰 후 수행 방법을 표시합니다.

필수 코드 확인은 [mcp\\_client.py](#) 67행(입력 메시지와 서버 실행), [mcp\\_server.py](#) 1617행(id·method), 2022행(초기화), 3443행(도구 요청과 결과)입니다. 구현 문법 전체를 풀이하지 않고 메시지의 역할과 입출력 대응을 확인합니다. 줄 번호는 이번 제공본 기준입니다.

**강사 참고·선택:** 서버 전체의 예외 처리, 표준 입출력, subprocess 구현과 모든 줄 해설. 전체 코드를 새로 작성하는 것은 필수가 아닙니다. 실제 AI 호스트 호출·인증·HTTP 연결은 이번 관찰 범위에 포함되지 않습니다.

## A13 · 따라 하기와 작성칸

# A13 · MCP의 호스트·클라이언트·서버

## 목적

React의 화면 구성과 MCP의 도구 통신을 같은 역할로 생각하지 않도록 경계를 구분합니다.

## 준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 역할 카드와 requests.jsonl의 초기화·첫 도구 요청, recorded\_responses.jsonl의 성공·미측정·isError를 필수로 읽습니다. 전체 구현·모든 줄 해설은 강사 참고·선택입니다.

입력: [role\\_cards.md](#), [mcp\\_scope.md](#), [mcp\\_client.py](#), [mcp\\_server.py](#), [A13\\_input.md](#)

작업: [A13\\_notes.md](#)

## 실제 입력

제공 클라이언트는 get\_reading이라는 읽기 도구를 요청하고 서버는 가상 센서 기록을 반환합니다.

## 단계

1. 역할 카드의 네 요소 읽기
2. 화면 역할과 도구 통신 역할 나누기
3. 클라이언트·서버·데이터 파일 연결하기
4. 실제 AI 호출이 없는 조건 쓰기

## 예상 결과

역할과 실제 실행 범위 표

## 확인 방법

MCP 서버가 웹 화면을 자동으로 만드는 것은 왜 아닌가요?

## 오류 해결

MCP를 모든 AI 서비스의 동의어로 보지 않고 오늘 메시지를 주고 받는 두 프로그램을 정확히 지정합니다.

## 힌트

MCP는 AI 앱이 외부 도구나 자료를 정해진 메시지 형식으로 사용하도록 연결하는 프로토콜입니다. 서버는 기능을 제공하고 클라이언트는 요청을 전달합니다.

## 빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

## 학생 답칸

- 개념 설명: MCP 서버가 웹 화면을 자동으로 만드는 것은 왜 아닌가요? → \_\_\_\_\_
- 입력 파일과 내가 바꾼 위치: \_\_\_\_\_
- 실행 전 예상 결과: \_\_\_\_\_
- 실제 관찰 결과와 확인 방법: \_\_\_\_\_
- 예상과 달랐던 점·복구·남은 질문: \_\_\_\_\_

## MCP 필수 읽기와 참고 범위

**필수:** 역할 카드에서 호스트·클라이언트·서버·데이터를 구별합니다. `requests.jsonl` 첫 세 줄의 초기화·알림·도구 목록과 네 번째 줄의 `method/id/params`를 읽습니다. `recorded_responses.json`에서 S01의 숫자, S02의 미측정, S99의 `isError`를 대조합니다. 제공 클라이언트 실행 또는 강사 실행 관찰 후 수행 방법을 표시합니다.

필수 코드 확인은 `mcp_client.py` 67행(입력 메시지와 서버 실행), `mcp_server.py` 1617행(id·method), 2022행(초기화), 3443행(도구 요청과 결과)입니다. 구현 문법 전체를 풀이하지 않고 메시지의 역할과 입출력 대응을 확인합니다. 줄 번호는 이번 제공본 기준입니다.

**강사 참고·선택:** 서버 전체의 예외 처리, 표준 입출력, subprocess 구현과 모든 줄 해설. 전체 코드를 새로 작성하는 것은 필수가 아닙니다. 실제 AI 호스트 호출·인증·HTTP 연결은 이번 관찰 범위에 포함되지 않습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: \_\_\_\_\_

실제 결과·근거: \_\_\_\_\_

다음에 확인할 것: \_\_\_\_\_

## A15 · 실제 STUDIO 도구 호출

**뜻과 비유.** STUDIO는 한 프로그램이 다른 프로그램의 표준 입력과 표준 출력으로 메시지를 교환하는 방법입니다. 네트워크 계정 없이 로컬 자식 프로세스를 실행할 수 있습니다.

**왜 필요한가.** 저장된 예시 문장을 읽는 것과 실제 서버가 응답한 기록을 구분하면서 작은 프로토콜을 관찰합니다.

**예를 따라 읽기.** `python code/mcp_client.py`는 서버를 실행해 S01의 22.5, S02의 null, S99의 도구 오류를 받습니다.

**햇갈리는 반례.** S02의 null은 통신 실패가 아닙니다. S99의 `isError:true`와 값 없음은 서로 다릅니다.

**직접 적용.** 완성 클라이언트와 서버 줄 해설 읽기 → 클라이언트 명령 실행 또는 강사 결과 비교하기 → id3·4·5의 값과 오류 표시 찾기 → 응답 예와 실제 실행 여부를 분리 기록하기

**설명할 질문.** null 응답과 도구 오류 응답은 무엇이 다른가요?

**막혔을 때.** 서버를 단독 실행하면 입력을 기다릴 수 있습니다. 기본 활동은 제공 클라이언트 명령으로 실행합니다.

입력·답변·확인 순서는 [실습지의 A15](#)와 [활동 원문](#)에 있습니다.

## MCP 필수 읽기와 참고 범위

**필수:** 역할 카드에서 호스트·클라이언트·서버·데이터를 구별합니다. [requests.jsonl](#) 첫 세 줄의 초기화·알림·도구 목록과 네 번째 줄의 [method/id/params](#)를 읽습니다. [recorded\\_responses.json](#)에서 S01의 숫자, S02의 미측정, S99의 [isError](#)를 대조합니다. 제공 클라이언트 실행 또는 강사 실행 관찰 후 수행 방법을 표시합니다.

필수 코드 확인은 [mcp\\_client.py](#) 67행(입력 메시지와 서버 실행), [mcp\\_server.py](#) 1617행(id·method), 2022행(초기화), 3443행(도구 요청과 결과)입니다. 구현 문법 전체를 풀이하지 않고 메시지의 역할과 입출력 대응을 확인합니다. 줄 번호는 이번 제공본 기준입니다.

**강사 참고·선택:** 서버 전체의 예외 처리, 표준 입출력, subprocess 구현과 모든 줄 해설. 전체 코드를 새로 작성하는 것은 필수가 아닙니다. 실제 AI 호스트 호출·인증·HTTP 연결은 이번 관찰 범위에 포함되지 않습니다.

## A15 · 따라 하기와 작성칸

### A15 · 실제 STUDIO 도구 호출

#### 목적

저장된 예시 문장을 읽는 것과 실제 서버가 응답한 기록을 구분하면서 작은 프로토콜을 관찰합니다.

#### 준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 역할 카드와 [requests.jsonl](#)의 초기화·첫 도구 요청, [recorded\\_responses.json](#)의 성공·미측정·isError를 필수로 읽습니다. 전체 구현·모든 줄 해설은 강사 참고·선택입니다.

입력: [mcp\\_client.py](#), [mcp\\_server.py](#), [requests.jsonl](#), [readings.json](#), [A15\\_input.md](#)

작업: [A15\\_notes.md](#)

#### 실제 입력

python code/mcp\_client.py는 서버를 실행해 S01의 22.5, S02의 null, S99의 도구 오류를 받습니다.

#### 단계

1. 완성 클라이언트와 서버 줄 해설 읽기
2. 클라이언트 명령 실행 또는 강사 결과 비교하기
3. id3·4·5의 값과 오류 표시 찾기

#### 4. 응답 예와 실제 실행 여부를 분리 기록하기

### 예상 결과

성공·미측정·오류 호출 관찰 표

### 확인 방법

null 응답과 도구 오류 응답은 무엇이 다른가요?

### 오류 해결

서버를 단독 실행하면 입력을 기다릴 수 있습니다. 기본 활동은 제공 클라이언트 명령으로 실행합니다.

### 힌트

STDIO는 한 프로그램이 다른 프로그램의 표준 입력과 표준 출력으로 메시지를 교환하는 방법입니다. 네트워크 계정 없이 로컬 자식 프로세스를 실행할 수 있습니다.

### 빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

### 학생 답칸

- 개념 설명: null 응답과 도구 오류 응답은 무엇이 다른가요? → \_\_\_\_\_
- 입력 파일과 내가 바꾼 위치: \_\_\_\_\_
- 실행 전 예상 결과: \_\_\_\_\_
- 실제 관찰 결과와 확인 방법: \_\_\_\_\_
- 예상과 달랐던 점·복구·남은 질문: \_\_\_\_\_

### MCP 필수 읽기와 참고 범위

**필수:** 역할 카드에서 호스트·클라이언트·서버·데이터를 구별합니다. `requests.jsonl` 첫 세 줄의 초기화·알림·도구 목록과 네 번째 줄의 `method/id/params`를 읽습니다. `recorded_responses.json`에서 S01의 숫자, S02의 미측정, S99의 `isError`를 대조합니다. 제공 클라이언트 실행 또는 강사 실행 관찰 후 수행 방법을 표시합니다.

필수 코드 확인은 `mcp_client.py` 67행(입력 메시지와 서버 실행), `mcp_server.py` 1617행(`id·method`), 2022행(초기화), 3443행(도구 요청과 결과)입니다. 구현 문법 전체를 풀이하지 않고 메시지의 역할과 입출력 대응을 확인합니다. 줄 번호는 이번 제공본 기준입니다.

**강사 참고·선택:** 서버 전체의 예외 처리, 표준 입출력, `subprocess` 구현과 모든 줄 해설. 전체 코드를 새로 작성하는 것은 필수가 아닙니다. 실제 AI 호스트 호출·인증·HTTP 연결은 이번 관찰 범위에 포함되지 않습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: \_\_\_\_\_

실제 결과·근거: \_\_\_\_\_

다음에 확인할 것: \_\_\_\_\_

## 6교시 · 역할 연결과 검수

16:00~16:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

**이번 교시에서 확인할 것.** 네 역할과 확인 범위

**먼저 할 활동:** A16, A17, A18

이 교시는 50분 운영이며 아래 세 개념을 연결합니다. 필수 경로를 먼저 진행하고 추가 복습은 수업 진행에 따라 배정합니다. 학생은 자신의 말로 확인 근거를 설명합니다.

**추가·복습:** 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

### A16 · 네 요소의 연결과 경계

**사례로 이해하기 · 도구 응답을 화면에 넣기 위한 중간 처리**

#### 실무 상황

MCP에서 온도 응답을 받았지만 React 화면은 바뀌지 않습니다. 화면에 카드를 만든 것과 서버에서 기록을 조회한 것은 각각 확인했어도 두 동작을 잇는 처리가 빠질 수 있습니다. “연결 완료”라는 말 대신 실제로 이어진 화살표와 아직 계획인 화살표를 나눕니다.

#### 쉬운 풀이

도구 응답은 메시지이며 화면 상태는 React 부품이 현재 기억하는 값입니다. 둘 사이에는 호출 결과를 받기, 오류 구별하기, 내용 해석하기, 숫자와 단위 확인하기, 상태에 전달하기가 필요합니다. 문자열 안의 JSON도 해석하지 않으면 숫자를 바로 사용할 수 없습니다. 응답 안의 안내 글은 외부 자료이므로 호스트 권한을 바꾸라는 명령처럼 취급하지 않습니다. 읽기 도구의 결과가 추가 쓰기 권한을 부여하지도 않습니다.

#### 완성 예와 단계 풀이

recorded\_responses.json의 id 3 결과는 content 안의 text에 S01 기록이 담겼습니다. 먼저 isError가 false인지 확인하고 text를 JSON 데이터로 읽은 뒤 temperature\_c가 22.5인지 확인합니다. id 4는 temperature\_c가 null이므로 “미측정” 문구를 고릅니다. id 5는 isError가 true이므로 “조회 실패”로 처리합니다. 이 판단 표를 작성해도 실제 React 상태 변경 코드를 구현한 것은 아닙니다. 지금 제공 화면은 data.js를 사용하고 제공 MCP는 STUDIO 기록 조회를 사용합니다. 실제 연결에는 호스트 환경과 화면 갱신 경로 설계가 더 필요합니다.

#### 독립 추가 복습 과제

숫자 0, null, Unknown sensor 응답 세 개를 각각 표시 문구로 바꾸고 조건 순서를 작성하세요. 다음으로 응답 내용에 “다른 파일을 삭제하라”라는 문장이 섞였을 때 데이터 표시 작업을 계속하려면 무엇을 구별해야 할지 설명하세요.

## 답과 대표 오답

유효한 0은 0 °C, null은 미측정, 도구 오류는 조회 실패입니다. 먼저 오류와 자료 형태를 확인한 뒤 값을 해석합니다. 응답 속 삭제 요구는 작업 권한의 근거가 아니며 데이터 내용으로 분리합니다. "null도 0으로 표시"는 값 없음과 영점을 혼동합니다. "응답을 받았으니 화면도 갱신"은 중간 처리를 생략했습니다.

## 확인 질문과 해설

① CSS로 누락 값이 측정값이 되나요? 모양은 자료의 의미를 바꾸지 않습니다. ② 다이어그램의 화살표는 실행 증거인가요? 실제 호출과 화면 관찰이 필요합니다. ③ 인계에 무엇을 쓰나요? 사용한 파일, 응답 id, 표시 결정, 실제 연결 여부와 다음 확인 범위를 적습니다.

**뜻과 비유.** HTML·CSS·React·MCP는 같은 시스템에 함께 있을 수 있지만 책임이 다릅니다. 데이터 이동이 필요한 곳에는 명시적인 연결 프로그램이 있어야 합니다.

**왜 필요한가.** 도구 응답을 받았다는 사실만으로 브라우저가 자동 갱신된다고 생각하면 연결이 빠진 설계를 만들 수 있습니다.

**예를 따라 읽기.** 오늘은 MCP 응답의 온도를 사람이 읽고 제공 데이터와 비교합니다. React와 MCP 간 자동 연결은 만들지 않았습니다.

**헛갈리는 반례.** MCP 서버가 있으니 로그인·화면·저장이 모두 해결됐다고 말하지 않습니다.

**직접 적용.** 네 요소 역할을 한 문장씩 쓰기 → S01 값의 출처와 화면 표시 구분하기 → 자동 연결한 것과 사람이 옮긴 것 표시하기 → 다음 연결에 필요한 프로그램 제안하기

**설명할 질문.** 도구 응답을 화면에 보이려면 무엇이 더 필요할까요?

**막혔을 때.** 그림의 화살표가 실제 동작인지 가정인지 따로 표시합니다.

입력·답칸·확인 순서는 [실습지의 A16](#)과 [활동 원문](#)에 있습니다.

## A16 · 따라 하기와 작성칸

### A16 · 네 요소의 연결과 경계

#### 목적

도구 응답을 받았다는 사실만으로 브라우저가 자동 갱신된다고 생각하면 연결이 빠진 설계를 만들 수 있습니다.

#### 준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [role\\_cards.md](#), [mcp\\_scope.md](#), [A16\\_input.md](#)

작업: [A16\\_notes.md](#)

#### 실제 입력

오늘은 MCP 응답의 온도를 사람이 읽고 제공 데이터와 비교합니다. React와 MCP 간 자동 연결은 만들지 않았습니다.

## 단계

1. 네 요소 역할을 한 문장씩 쓰기
2. S01 값의 출처와 화면 표시 구분하기
3. 자동 연결한 것과 사람이 옮긴 것 표시하기
4. 다음 연결에 필요한 프로그램 제안하기

## 예상 결과

역할과 미구현 연결을 표시한 설명

## 확인 방법

도구 응답을 화면에 보이려면 무엇이 더 필요할까요?

## 오류 해결

그림의 화살표가 실제 동작인지 가정인지 따로 표시합니다.

## 힌트

HTML·CSS·React·MCP는 같은 시스템에 함께 있을 수 있지만 책임이 다릅니다. 데이터 이동이 필요한 곳에는 명시적인 연결 프로그램이 있어야 합니다.

## 빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

## 학생 답칸

- 개념 설명: 도구 응답을 화면에 보이려면 무엇이 더 필요할까요? → \_\_\_\_\_
- 입력 파일과 내가 바꾼 위치: \_\_\_\_\_
- 실행 전 예상 결과: \_\_\_\_\_
- 실제 관찰 결과와 확인 방법: \_\_\_\_\_
- 예상과 달랐던 점·복구·남은 질문: \_\_\_\_\_

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: \_\_\_\_\_

실제 결과·근거: \_\_\_\_\_

다음에 확인할 것: \_\_\_\_\_

## A17 · 작은 변경의 동료 확인

**뜻과 비유.** 변경 검수는 입력한 값·바뀐 파일·예상 화면·실제 결과를 짝이 다시 확인하는 것입니다. 내용·모양·데이터·통신을 각기 다른 근거로 봅니다.

**왜 필요한가.** 한 화면이 예뻐졌어도 도구 요청 입력이 맞거나 자료형이 보존됐다는 증거는 아닙니다.

**예를 따라 읽기.** HTML 제목,CSS 색,React 장소·온도,MCP sensor\_id를 각각 한 줄의 변경 기록으로 남깁니다.

**헛갈리는 반례.** CSS 색 변경을 센서 상태로 기록하거나 모의 버튼을 실제 측정이라고 부르지 않습니다.

**직접 적용.** 작업 파일과 관찰 표 교환하기 → 네 역할 중 각 한 개 근거 확인하기 → 새로고침과 클릭 결과 구분하기 → 설명이 모호한 한 곳 보완하기

**설명할 질문.** 같은 화면에서 서로 다른 역할의 변경을 어떻게 확인하나요?

**막혔을 때.** 짝의 코드를 바로 수정하지 말고 대상 파일과 기대한 결과를 먼저 설명합니다.

입력·답칸·확인 순서는 [실습지의 A17](#)와 [활동 원문](#)에 있습니다.

## A17 · 따라 하기와 작성칸

### A17 · 작은 변경의 동료 확인

#### 목적

한 화면이 예뻐졌어도 도구 요청 입력이 맞거나 자료형이 보존됐다는 증거는 아닙니다.

#### 준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [index.html](#), [style.css](#), [data.js](#), [requests.jsonl](#), [A17\\_input.md](#)

작업: [A17\\_notes.md](#), [index.html](#), [style.css](#), [data.js](#)

#### 실제 입력

HTML 제목,CSS 색,React 장소·온도,MCP sensor\_id를 각각 한 줄의 변경 기록으로 남깁니다.

#### 단계

1. 작업 파일과 관찰 표 교환하기
2. 네 역할 중 각 한 개 근거 확인하기
3. 새로고침과 클릭 결과 구분하기
4. 설명이 모호한 한 곳 보완하기

#### 예상 결과

네 요소별 입력·결과 검토 표

#### 확인 방법

같은 화면에서 서로 다른 역할의 변경을 어떻게 확인하나요?

## 오류 해결

짜의 코드를 바로 수정하지 말고 대상 파일과 기대한 결과를 먼저 설명합니다.

## 힌트

변경 검수는 입력한 값·바뀐 파일·예상 화면·실제 결과를 짜이 다시 확인하는 것입니다. 내용·모양·데이터·통신을 각기 다른 근거로 봅니다.

## 빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

## 학생 답칸

- 개념 설명: 같은 화면에서 서로 다른 역할의 변경을 어떻게 확인하나요? → \_\_\_\_\_
- 입력 파일과 내가 바꾼 위치: \_\_\_\_\_
- 실행 전 예상 결과: \_\_\_\_\_
- 실제 관찰 결과와 확인 방법: \_\_\_\_\_
- 예상과 달랐던 점·복구·남은 질문: \_\_\_\_\_

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: \_\_\_\_\_

실제 결과·근거: \_\_\_\_\_

다음에 확인할 것: \_\_\_\_\_

## A18 · 결과 설명과 선택 확장

**뜻과 비유.** 학습 결과는 수정한 작은 부분의 이유와 결과를 설명하는 것입니다. 전체 앱이나 MCP 서버를 독립 작성하는 능력을 오늘의 필수 목표로 삼지 않습니다.

**왜 필요한가.** 네 기술의 이름만 기억하는 것보다 어느 문제를 어떤 요소가 맡는지 말할 수 있어야 다음 대시보드로 연결됩니다.

**예를 따라 읽기.** 필수 결과는 제목·색·React 데이터 수정과 MCP 요청·응답 한 쌍의 설명입니다.

**헛갈리는 반례.** 패키지 설치를 못 했다고 개념 설명과 제공 번들 수정까지 실패로 판정하지 않습니다.

**직접 적용.** 수정한 파일 목록 정리하기 → 실제로 실행한 경로 쓰기 → 네 역할에 대한 새 질문 답하기 → 남은 환경·자동 연결 조건을 인계하기

**설명할 질문.** React 상태·CSS 색·MCP 응답은 각각 무엇을 바꾸나요?

**막혔을 때.** 설치나 외부 호스트 연결이 막히면 오프라인 번들과 저장된 응답으로 동일 개념을 확인하고 실행 범위를 명시합니다.

입력·답칸·확인 순서는 [실습지의 A18](#)와 [활동 원문](#)에 있습니다.

## 공식 근거와 원본

확인일 2026-09-26. 정의는 짧게 요약하고 활동·데이터·설명은 수업용으로 새로 작성했습니다.

- 원본 Track A / A3.pptx, 슬라이드 164-189: HTML/CSS/JavaScript 참고. React 본문과 MCP 실제 예제는 신규 제작했습니다. 원본은 강사용 로컬 자료에 보존합니다.

- [HTML 기초](#): 태그·요소·속성의 의미
- [DOM 선택](#): 요소 선택 동작
- [React 상태](#): 상태와 setter 동작
- [React 화면 시작](#): DOM 루트에 컴포넌트 렌더링
- [MCP 수명주기](#): 명시 버전의 initialize와 initialized
- [MCP STUDIO](#): 한 줄 JSON-RPC·표준입출력
- [MCP 도구](#): tools/list와 tools/call·입력 스키마

### 개념 보충 공식 근거 (2026-10-05 확인)

- [React JSX: 화면 구조를 적는 문법](#)
- [MDN 상자 모델](#)
- [MCP 2025-11-25 구조와 역할](#)
- [MCP 도구와 사용자 통제](#)

## A18 · 따라 하기와 작성칸

### A18 · 결과 설명과 선택 확장

#### 목적

네 기술의 이름만 기억하는 것보다 어느 문제를 어떤 요소가 맡는지 말할 수 있어야 다음 대시보드로 연결됩니다.

#### 준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [role\\_cards.md](#), [mcp\\_scope.md](#), [A18\\_input.md](#)

작업: [A18\\_notes.md](#)

#### 실제 입력

필수 결과는 제목·색·React 데이터 수정과 MCP 요청·응답 한 쌍의 설명입니다.

## 단계

1. 수정한 파일 목록 정리하기
2. 실제로 실행한 경로 쓰기
3. 네 역할에 대한 새 질문 답하기
4. 남은 환경·자동 연결 조건을 인계하기

## 예상 결과

수정 묶음과 역할·검증·미확인 범위 보고

## 확인 방법

React 상태·CSS 색·MCP 응답은 각각 무엇을 바꾸나요?

## 오류 해결

설치나 외부 호스트 연결이 막히면 오프라인 번들과 저장된 응답으로 동일 개념을 확인하고 실행 범위를 명시합니다.

## 힌트

학습 결과는 수정한 작은 부분의 이유와 결과를 설명하는 것입니다. 전체 앱이나 MCP 서버를 독립 작성하는 능력을 오늘의 필수 목표로 삼지 않습니다.

## 빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

## 학생 답칸

- 개념 설명: React 상태·CSS 색·MCP 응답은 각각 무엇을 바꾸나요? → \_\_\_\_\_
- 입력 파일과 내가 바꾼 위치: \_\_\_\_\_
- 실행 전 예상 결과: \_\_\_\_\_
- 실제 관찰 결과와 확인 방법: \_\_\_\_\_
- 예상과 달랐던 점·복구·남은 질문: \_\_\_\_\_

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: \_\_\_\_\_

실제 결과·근거: \_\_\_\_\_

다음에 확인할 것: \_\_\_\_\_

## 추가 활동 · 필수 경로를 마친 뒤

아래는 기존 웹의 개념·활동입니다. 수업 시간의 필수 결과를 먼저 확인한 뒤 선택하거나 복습에 사용합니다. 기존 웹의 활동 별 분량 배정은 이번 PDF의 50분 교시 시간표와 별도로 보존된 설계입니다.

### A14 · JSON-RPC 요청과 초기화

**뜻과 비유.** JSON-RPC 메시지는 method로 요청 종류를, id로 요청과 응답의 대응을 나타냅니다. 오늘의 MCP 예제는 2025-11-25규격 버전에 고정한 초기화 절차를 사용합니다.

**왜 필요한가.** 버전과 기능을 먼저 맞추면 어떤 도구 통신을 기대할지 확인할 수 있습니다. 최신 전체 규격 구현과 교육용 일부 구현을 구분합니다.

**예를 따라 읽기.** initialize 응답 뒤 notifications/initialized를 보내고 tools/list로 get\_reading의 입력 규칙을 읽습니다.

**헛갈리는 반례.** id가 없는 알림은 일반 요청과 달리 같은 id의 결과 응답을 기다리는 메시지가 아닙니다.

**직접 적용.** requests.jsonl의 첫 세 메시지 읽기 → method·id·params 표시하기 → 초기화 요청과 알림 구별하기 → tools/list에서 입력 필드 찾기

**설명할 질문.** 초기화 알림과 도구 요청은 어떻게 구분하나요?

**막혔을 때.** JSONL은 메시지 한 개당 한 줄입니다. 예쁘게 여러 줄로 바꾸면 이 STDIO 입력 규칙과 맞지 않습니다.

입력·답변·확인 순서는 [실습지의 A14](#)와 [활동 원문](#)에 있습니다.

### MCP 필수 읽기와 참고 범위

**필수:** 역할 카드에서 호스트·클라이언트·서버·데이터를 구별합니다. `requests.jsonl` 첫 세 줄의 초기화·알림·도구 목록과 네 번째 줄의 `method/id/params`를 읽습니다. `recorded_responses.json`에서 S01의 숫자, S02의 미측정, S99의 `isError`를 대조합니다. 제공 클라이언트 실행 또는 강사 실행 관찰 후 수행 방법을 표시합니다.

필수 코드 확인은 `mcp_client.py` 67행(입력 메시지와 서버 실행), `mcp_server.py` 1617행(id·method), 2022행(초기화), 3443행(도구 요청과 결과)입니다. 구현 문법 전체를 풀이하지 않고 메시지의 역할과 입출력 대응을 확인합니다. 줄 번호는 이번 제공본 기준입니다.

**강사 참고·선택:** 서버 전체의 예외 처리, 표준 입출력, subprocess 구현과 모든 줄 해설. 전체 코드를 새로 작성하는 것은 필수가 아닙니다. 실제 AI 호스트 호출·인증·HTTP 연결은 이번 관찰 범위에 포함되지 않습니다.

## A14 · JSON-RPC 요청과 초기화

### 목적

버전과 기능을 먼저 맞추면 어떤 도구 통신을 기대할지 확인할 수 있습니다. 최신 전체 규격 구현과 교육용 일부 구현을 구분합니다.

## 준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 역할 카드와 requests.json의 초기화·첫 도구 요청, recorded\_responses.json의 성공·미측정·isError를 필수로 읽습니다. 전체 구현·모든 줄 해설은 강사 참고·선택입니다.

입력: [requests.jsonl](#), [mcp\\_scope.md](#), [A14\\_input.md](#)

작업: [A14\\_notes.md](#)

## 실제 입력

initialize 응답 뒤 notifications/initialized를 보내고 tools/list로 get\_reading의 입력 규칙을 읽습니다.

## 단계

1. requests.jsonl의 첫 세 메시지 읽기
2. method·id·params 표시하기
3. 초기화 요청과 알림 구별하기
4. tools/list에서 입력 필드 찾기

## 예상 결과

세 메시지의 역할과 대응 기록

## 확인 방법

초기화 알림과 도구 요청은 어떻게 구분하나요?

## 오류 해결

JSONL은 메시지 한 개당 한 줄입니다. 예쁘게 여러 줄로 바꾸면 이 STDIO 입력 규칙과 맞지 않습니다.

## 힌트

JSON-RPC 메시지는 method로 요청 종류를, id로 요청과 응답의 대응을 나타냅니다. 오늘의 MCP 예제는 2025-11-25 규격 버전에 고정한 초기화 절차를 사용합니다.

## 빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

## 학생 답칸

- 개념 설명: 초기화 알림과 도구 요청은 어떻게 구분하나요? → \_\_\_\_\_
- 입력 파일과 내가 바꾼 위치: \_\_\_\_\_
- 실행 전 예상 결과: \_\_\_\_\_
- 실제 관찰 결과와 확인 방법: \_\_\_\_\_

- 예상과 달랐던 점·복구·남은 질문: \_\_\_\_\_

## MCP 필수 읽기와 참고 범위

**필수:** 역할 카드에서 호스트·클라이언트·서버·데이터를 구별합니다. `requests.jsonl` 첫 세 줄의 초기화·알림·도구 목록과 네 번째 줄의 `method/id/params`를 읽습니다. `recorded_responses.json`에서 S01의 숫자, S02의 미측정, S99의 `isError`를 대조합니다. 제공 클라이언트 실행 또는 강사 실행 관찰 후 수행 방법을 표시합니다.

필수 코드 확인은 `mcp_client.py` 67행(입력 메시지와 서버 실행), `mcp_server.py` 1617행(id·method), 2022행(초기화), 3443행(도구 요청과 결과)입니다. 구현 문법 전체를 풀이하지 않고 메시지의 역할과 입출력 대응을 확인합니다. 줄 번호는 이번 제공본 기준입니다.

**강사 참고·선택:** 서버 전체의 예외 처리, 표준 입출력, subprocess 구현과 모든 줄 해설. 전체 코드를 새로 작성하는 것은 필수가 아닙니다. 실제 AI 호스트 호출·인증·HTTP 연결은 이번 관찰 범위에 포함되지 않습니다.

# 용어와 코드 기호 찾아보기

## HTML/CSS/React/MCP 쉬운 용어

용어를 말한 뒤 예제의 실제 위치를 함께 짚습니다.

| 용어    | 뜻                               |
|-------|---------------------------------|
| HTML  | 제목·문단·버튼처럼 문서의 내용과 의미를 표시하는 형식  |
| 태그    | 요소를 표시하는 꺾쇠 안의 이름               |
| 속성    | 요소에 붙이는 이름과 값의 추가 정보            |
| CSS   | 내용의 색·크기·간격·배치를 정하는 규칙          |
| 선택자   | CSS나 JavaScript가 대상을 고르는 표현     |
| 이벤트   | 클릭처럼 프로그램이 반응할 수 있는 사건          |
| 컴포넌트  | 다시 쓸 수 있도록 묶은 화면 부품             |
| props | 부모가 컴포넌트에 전달하는 입력               |
| 상태    | 컴포넌트가 기억하며 바뀔 수 있는 값            |
| 번들    | 여러 소스와 라이브러리를 브라우저가 실행하도록 묶은 출력 |

| 용어        | 뜻                                       |
|-----------|-----------------------------------------|
| MCP       | AI 앱과 도구 제공 프로그램의 요청·응답을 연결하는 프로토콜      |
| STDIO     | 프로그램의 표준입력과 표준출력을 이용한 통신                |
| JSON-RPC  | 메서드·입력·id·결과를 JSON 메시지로 주고받는 형식         |
| MCP 호스트   | 사용자가 요청하는 AI 앱 전체로 연결과 도구 사용 흐름을 관리하는 쪽 |
| MCP 클라이언트 | 호스트 쪽에서 특정 서버와 메시지를 주고받는 연결 부분          |
| MCP 서버    | 도구와 입력 규칙을 제공하고 호출 결과를 돌려주는 프로그램        |

## 오늘 배운 내용 확인하기

## 2026-10-21 확인·평가

공식 수료 기준이 아닌 형성평가입니다. 개인 설치 성공만으로 개념 이해를 판단하지 않습니다. 학생은 직접 수정·예상·대조·설명을 수행합니다.

| 구간          | 확인 질문                               | 충족 근거                  |
|-------------|-------------------------------------|------------------------|
| 웹 문서의 내용    | 사용자에게 보이는 이름과 코드가 찾는 이름은 왜 구분하나요?   | 내용 수정과 내부 연결 이름의 차이 기록 |
| CSS의 모양과 읽기 | 정상 상태를 색만으로 표시하면 어떤 정보가 빠질까요?       | 화면 폭·키보드·상태 문구 검수 표    |
| 버튼과 데이터 변화  | 원복 후에도 같은 행동으로 확인해야 하는 이유는 무엇인가요?   | 한 오류의 증상·원인·복구 기록      |
| React 완성 부품 | 상태 변경과 파일 저장은 어떤 차이가 있나요?           | 상태 변화와 초기화 기록          |
| MCP 요청과 응답  | null 응답과 도구 오류 응답은 무엇이 다른가요?        | 성공·미측정·오류 호출 관찰 표      |
| 역할 연결과 검수   | React 상태·CSS 색·MCP 응답은 각각 무엇을 바꾸나요? | 수정 묶음과 역할·검증·미확인 범위 보고 |

## 최종 제출

- work의18활동 답칸과 직접 수정한 예제 복사본
- 바꾼 파일·입력·예상·실제 결과를 연결한 검수 기록
- 완료 범위·실제 실행 방식·아직 확인하지 않은 조건의 인수인계

## 판단 방법

스스로 설명함 / 예를 보고 설명함 / 다시 연습 필요를 각 구간에 기록합니다. 결과가 틀렸더라도 근거를 찾아 수정하고 다시 확인한 과정을 남깁니다. 관찰하지 않은 출력을 실제 실행으로 쓰지 않습니다. 빠른 학생의 확장은 필수 기준에 포함하지 않습니다.

## 개념 보충 전이 문제 · 독립 추가 복습

교시별 필수 설명을 다른 상황에 적용하는 추가 복습입니다. 50분 필수 경로에 모두 포함되어 완수된 것으로 처리하지 않습니다. 답안은 먼저 학생이 작성하고 이후 아래 해설을 대조합니다. 설치·계정 성공 여부와 판단 근거를 분리합니다.

### 전이 1 · 서로 다른 이름

**문제:** 탭=공장 A, 본문 제목=온도 확인, 버튼 글자=기록 조회로 바꾸고 내부 id check를 유지하려면 어느 태그 부분을 고치나요? 버튼 반응까지 생겼다고 보고할 수 있나요?

**해설·확인 기준:** title, h1의 내용, button의 태그 사이 문구를 바꾸며 id는 유지합니다. 동작 코드가 없으면 클릭 반응은 생기지 않습니다. 저장→같은 경로 열기→새로고침 확인까지 설명합니다.

### 전이 2 · 안쪽과 바깥 간격

**문제:** 버튼 글자 주변만 넓히고 다른 카드와의 거리는 유지하려 합니다. 속성을 선택하세요. 녹색 원만 있는 정상 표시는 어떻게 보완하나요?

**해설·확인 기준:** padding을 바꾸고 margin은 유지합니다. "정상: 가상 온도가 기준 미만입니다" 같은 글자를 함께 표시합니다. Tab으로 초점도 확인합니다. 색 자체가 데이터의 정확성 근거는 아닙니다.

### 전이 3 · 초기 표시와 클릭값

**문제:** HTML은 22.5, data.js는 26입니다. 열기 직후와 클릭 뒤 값은 무엇인가요? 버튼 id가 read인데 선택자가 #check 이면 어디를 고치고 어떻게 재검사하나요?

**해설·확인 기준:** 직후 22.5, 클릭 뒤 26입니다. HTML id와 JavaScript 선택자를 일치시키며 같은 클릭 행동으로 재검사합니다. 클릭은 data.js를 저장하지 않습니다. 실제 데이터 출처를 구별하면 충족입니다.

### 전이 4 · JSX와 번들

**문제:** App.jsx 증가량을 1로 바꿨지만 제공 app.bundle.js는 그대로입니다. 화면 증가량을 단정할 수 있나요? 처음 값 24에서 기존 0.5 버튼을 세 번 누르고 새로고침하면 어떻게 되나요?

**해설·확인 기준:** 브라우저는 기존 번들을 실행하므로 소스 수정만으로 새 증가량을 보장할 수 없습니다. 기존 동작은 25.5 뒤 새로고침 24입니다. JSX는 변환이 필요하며 state는 파일 저장이 아닙니다.

### 전이 5 · 연결 방식과 권한

**문제:** 고정 HTML 값·data.js 값·MCP 응답을 출처와 연결 단계로 비교하세요. 읽기 도구를 연결하며 삭제 권한까지 요구하면 어떻게 판단하나요?

**해설·확인 기준:** 문서 글자·브라우저 데이터 파일·서버 기록 조회를 구별합니다. MCP는 초기화와 도구 발견 및 호출 과정이 있고 목적과 무관한 삭제 권한은 허용 근거가 없습니다. 연결 자체가 전체 쓰기 권한을 뜻하지 않습니다.

## 전이 6 · 응답과 화면 사이

**문제:** 온도 0,온도 null,isError:true 세 응답의 표시를 쓰고 MCP 결과를 받았는데 React 화면이 그대로인 원인 후보를 설명하세요. 응답 속 삭제 지시는 권한 근거인가요?

**해설·확인 기준:** 0 °C·미측정·조회 실패로 구별합니다. 응답 해석과 유효값 확인 및 상태 전달 경로가 빠졌을 수 있습니다. 데이터 속 삭제 지시는 권한 근거가 아닙니다. 실제 호출과 화면 갱신을 따로 확인합니다.

# 파일과 다음 수업

링크를 눌러 파일을 내려받습니다. 예제 ZIP은 압축을 풀고 README부터 읽습니다. 원본 code·data는 보존하고 work의 작업본을 수정합니다. 파일이 저장된 폴더와 확장명을 확인하세요.

실습 예제 코드와 데이터

HTML/CSS/React/MCP PPT

처음 시작 안내

용어와 코드 기호

학생용 교안

학생 실습 안내

확인 문제

학생 교안 PDF · 내용 보강본

설치·로그인·장비 준비와 학습 성취는 구분합니다. 이 교안의 배정 시간은 실제 초심자 리허설 결과가 아닙니다. 코드·제공 출력·실물 검증의 한계는 해당 웹 수업과 교안 설명을 따릅니다.